

Greedy Algorithms:

Introduction, Elements of Greedy Strategy

- Minimum Spanning Tree: Kruskal's & Prim's Algorithm, Dijkstra's Algorithm, Knapsack Problem, Activity Selection Problem, Huffman Codes

❖ Greedy Technique:- (Shortcut)

- Most of the problems in greedy contain n-number of inputs and our requirement is finding a subset which will satisfy our conditions and optimizes our goal.
- **Basics of GT**
- Solution Space : →
- Feasible Solution : →
- Optimal Solution : →
- **Solution Space**
- Set of all possible solutions over given n-inputs is known as solution space.
- **Example:** → G1, G2, G3, G [500 C 10]
- **Feasible Solution**
- Set of solutions which will satisfy our conditions.
Eligibility = 75%
- **Example:** → G1, G2, G3, G [50 C 10]
- **Optimal Solution**
- It need not be unique.
Example: → G1
- It is one of the feasible solutions which optimizes our goal.

❖Control Abstraction for Greedy Method:-

- Algorithm Greedy Method (a, n)
- {
- // a is an array of n inputs
- Solution: = $\emptyset$;
- for i: =0 to n do
- {
- s: = select (a);
- if (feasible (Solution, s)) then
- {
- Solution: = union (Solution, s);
- }
- else
- reject (); // if solution is not feasible reject it.
- }
- return solution;
- }

❖ In greedy method there are three important activities.

- 1. A selection of solution from the given input domain is performed, i.e. $s := \text{select}(a)$.
- 2. The feasibility of the solution is performed, by using feasible '(solution, s)' and then all feasible solutions are obtained.
- 3. From the set of feasible solutions, the particular solution that minimizes or maximizes the given objection function is obtained. Such a solution is called optimal solution.
- Example:-
- A child buys a candy 42 rupees and gives a 100 note to the cashier. Then the cashier wishes to return change using the fewest number of coins. Assume that the cashier has Rs.1, Rs. 5 and Rs. 10 coins.
- This problem can be solved using the greedy method.

❖ Disadvantages of Greedy Algorithm:-

- **No guarantee of global optimum**

Greedy choices are based on local benefit, which may lead to suboptimal global results.

- **Not suitable for all problems**

Only works when the problem has **greedy-choice property** and **optimal substructure**.

- **Hard to identify applicability**

It's often **non-obvious** whether a greedy approach will work correctly for a given problem.

- **Lack of backtracking**

Once a choice is made, greedy algorithms **don't reconsider** — this can lead to poor results.

- **Can fail in complex dependency problems**

Problems where future choices depend heavily on earlier ones often require more advanced methods (e.g., dynamic programming).

❖ **Application of Greedy:-**

- Job sequencing with deadlines ←
- Knapsack problem ←
- Huffman coding ←
- Optimal merge patterns ←
- Minimum cost spanning tree (MST) ←
- Single source shortest path ←

❖ **Knapsack problem or Fractional Knapsack:-**

- APPLICATION - KNAPSACK PROBLEM
- In this problem the objective is to fill the knapsack with items to get maximum benefit (value or profit) without crossing the weight capacity of the knapsack. And we are also allowed to take an item in fractional part.
- Points to remember:-
- In this problem we have a Knapsack that has a weight limit W
- There are items $i_1, i_2, \dots, i_n$ each having weight $w_1, w_2, \dots, w_n$ and some benefit (value or profit) associated with it $v_1, v_2, \dots, v_n$
- Our objective is to maximize the benefit such that the total weight inside the knapsack is at most W . And we are also allowed to take an item in fractional part.

❖ **Example(1)**

- **$M = 31(\text{kg})$ (Capacity of the knapsack or bag)**
- $\{12, 200\}$
- $\{8, 180\}$
- $\{14, 300\}$
- $\{5, 150\}$
- What objects & in which fraction should take so that maximum profit?

❖Solution:-

- **Step 1:-**

- **Obj1:**
 $12\text{kg} \rightarrow 200$
 $1\text{ kg} \rightarrow 200 / 12 = \mathbf{16.66}$

- **Obj3:**
 $14 \rightarrow 300$
 $1 \rightarrow 300 / 14 = \mathbf{21.42}$

- **Obj2:**
 $8 \rightarrow 180$
 $1 \rightarrow 180 / 8 = \mathbf{22.5}$

- **Obj4:**
 $5 \rightarrow 150 = \mathbf{30}$

❖Explanation:-

- **Step 1:** For every object, 1-unit would have profit. Just calculate.
- **Step 2:** After that, which one is larger, just take first, further continue:
 - next larger take
 - next larger take
- **Until capacity of knapsack is over**

x1 x2 x3 x4

4/12 1 1 1

(1/3)

Fraction taken $\rightarrow$ x1

Fully taken $\rightarrow$ x2,x3,x4

Step

Initial

Profit

P = 0

$$0 + 150 \times 1 = 150$$

$$150 + 180 \times 1 = 330$$

$$330 + 300 \times 1 = 630$$

$$630 + 200 \times 4/12 = 696.67$$

It is maximum profit

Capacity

$$m = 31$$

$$31 - 5 \times 1 = 26$$

$$26 - 8 \times 1 = 18$$

$$18 - 14 \times 1 = 4$$

$$4 - 12 \times 4/12 = 0$$

❖ Step 3:

A: $(1/3, 1, 1, 1) \rightarrow$ Feasible , Optimal also $\rightarrow$ Max Profit 

• B: $1/2, 1, 1, 0 \rightarrow$ Feasible , but not max profit 

• C: $0, 0, 0, 0 \rightarrow$ Feasible , but zero profit 

❖ Notice:-

This solution will get after giving priority to both profit and weight

❖ Example(2) where $M = 54$, $N = 7$

Object	obj1	obj2	obj3	obj4	obj5	obj6	obj7
Profit	75	200	150	50	38	90	64
Weight	10	20	15	3	2	7	5

- Step(1):- P_i / W_i
- || 7.5 | 10 | 10 | 16.77 | 19 | 12.85 | 12.8 |
- **Step(2):-Sorting:**
Descending order:
- 19,16.77,12.85,12.8,10,10,7.5
- **Step(3):- Take one by one object into Knapsack until capacity become 0.**

x_1	x_2	x_3	x_4	x_5	x_6	x_7
2/10	1	1	1	1	1	1

- Initially Profit $P = 0$, Bag/ Weight/ Knapsack(m) = 54

- $0 + 38 \Rightarrow 38$ $54 - 2 \times 1 \Rightarrow 52$
- $38 + 50 \Rightarrow 88$ $52 - 3 \Rightarrow 49$
- $88 + 90 \Rightarrow 178$ $49 - 7 \Rightarrow 42$
- $178 + 64 \Rightarrow 242$ $42 - 5 \Rightarrow 37$
- $242 + 200 \Rightarrow 442$ $37 - 20 \Rightarrow 17$
- $442 + 150 \Rightarrow 592$ $17 - 15 \Rightarrow 2$
- $592 + 75 \times 2 \div 10 \Rightarrow 607$ $2 - 10 \times 2 \div 10 \Rightarrow 0$

✓ SO, Maximum Profit = 607 //

```
FractionalKnapsack(W, weights[], profits[], n)
  FOR i ← 0 TO n - 1 DO
    ratio[i] ← profits[i] / weights[i]
  END FOR
  Sort items by ratio[] in descending order
  totalProfit ← 0
  remainingCapacity ← W

  FOR i ← 0 TO n - 1 DO
    IF weights[i] ≤ remainingCapacity THEN
      totalProfit ← totalProfit + profits[i]
      remainingCapacity ← remainingCapacity - weights[i]
    ELSE
      fraction ← remainingCapacity / weights[i]
      totalProfit ← totalProfit + profits[i] * fraction
      BREAK
    END IF
  END FOR

  RETURN totalProfit
```

➤ **Note(1):-**

Greedy knapsack will give optimal solution
by giving priority to both profit & weight

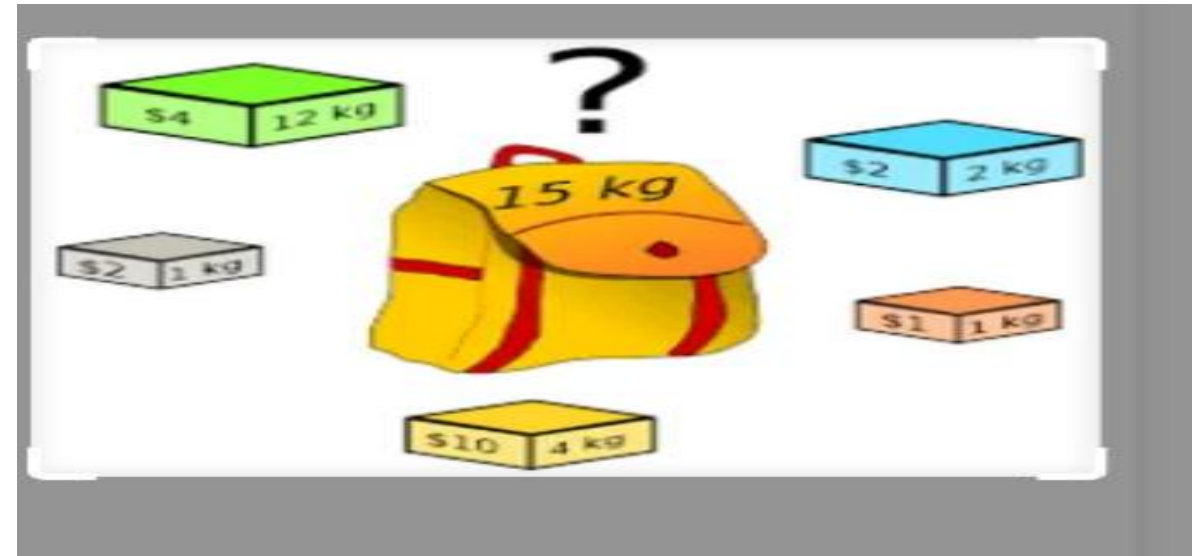
- **Time Complexity of Knapsack $\Rightarrow O(n \log n) \Rightarrow$ Efficient Case/best case**
- **Time C. of knapsack problem, if we q.s. $\Rightarrow O(n^2) \Rightarrow$ Worst Case**
T.C. of K.P if we s.s. $\Rightarrow O(n^2) \Rightarrow$ every Case

➤ **Note(2):-**

T.C. of K.P if objects are already sorted by decreasing order of P_i/W_i
 $\Rightarrow$ then step (1) & (2) **Not required**
So, only required step (3) $\Rightarrow O(n)$

❖ **Example(3):-**

- ✓ Find the Optimal Solution?
- ✓ Find the Maximum Profit?



❖ Huffman Coding:-

- Huffman coding is a lossless data compression algorithm.
- In this algorithm, a variable-length code is assigned to input different characters.
- The code length is related to how frequently characters are used.
- Most frequent characters have the smallest codes and longer codes for least frequent characters.
- There are mainly two parts.
- First one to create a Huffman tree, and another one to traverse the tree to find codes.

❖ Example(1):-

- {a = 45 b = 1
- c = 3 d = 20
- e = 30 f = 1} // m = 100 Char.

S → (Sender) -----> R → (Receiver)

□ ASCII – 8 bits

- $a = 45 * 8$

- $b = 1 * 8$

- $c = 3 * 8$

- $d = 20 * 8$

- $e = 30 * 8$

- $f = 1 * 8$

- **100 char = 800 bits**

- 1 char = $800 / 100$ b/c

- Avg = 8 b/c**

-

- Can we compress??**

- (Yes)**

□ ASCII – 3 bits

- $a = 45 * 3$

- $b = 1 * 3$

- $c = 3 * 3$

- $d = 20 * 3$

- $e = 30 * 3$

- $f = 1 * 3$

- **100 char = 300 bits**

- $1 \text{ char} = 300 / 100$

- Avg = 3 b/c**

-

- Can we compress??**

- (Yes), i.e., called "**Huffman coding**"

➤ **Note:-**
"Huffman Coding" follows **non-uniformity**.
It treats different people differently, not uniformly.

- ↔ **Huffman Coding** ↔
(It is a data compression technique) ↔ (It is data encoding)

- 
Non-uniform

More frequency → Less Bits → Less space

✓ Solution:- ex(1)

✓ M = 100 Char.

✓ a = 45 ✗

✓ b = 1 ✗

✓ c = 3 ✗

✓ d = 20 ✗

✓ e = 30 ✗

✓ f = 1 ✗

✓ t1 = 2	
✓ t2 = 5	
✓ t3 = 25	
✓ t4 = 55	

- **Step 1:** Just take 2 minimum first elements & store/Add it.
- **Step 2:** It again rewrites in the box (new value) with min.
- **Step 3:** Those procedures over, just cancel from box ✗
- **Note:-** Always take **1st min in left side**, 2nd min in right → step 4.

- 100(m) //finally done, so stop it

0/ \1

(a)45 55(t4)

0/ \1

(t3) 25 30(e)

0/ \1

(t2)5 20(d)

0/ \1

(t1)2 3 (c) // this t1 will store in box

0/ \1

b f //here cancel from box b & f

1 1

- **Step 4:**
- After Tree Construction is over, just go on each & every left & right edge.
- **Left edge** given $\rightarrow 0$
Right edge given $\rightarrow 1$
- **Note:**
 Top of people contain big one & bottom of people contain smaller.
- **Step 5:**
- After that just see tree & try to follow path from root to that particular element
- Huffman Code for every char:
 - a = 0
 - b = 10000
 - c = 1001
 - d = 101
 - e = 11
 - f = 10001
- a = 45, b = 5
 c = 3, d = 20
 e = 30, f = 1
- Note:- from above more frequency = less no. of length & Vice-Versa

- **Step 6:**
- **Total = $45 \times 1 + 1 \times 5 + 3 \times 4 + 20 \times 3 + 30 \times 2 + 1 \times 5$
= 187 bits**

- **Step 7:**

- **Avg bits/Char**

- **100 char = 187 bit**

- **1 char = $187 / 100$**

- **So, Avg = $187 / 100 = 1.87$ b/c**

➤ **Note:-1**

This 100 came from root-node.

→ So, whatever got there, just divide by that value only.

➤ **Note:-2**

The Huffman code cannot be prefix of another of same code.

→ i.e. It's called **Optimal Prefix Notation**.

- **Time Complexity:-**
- In given tree, total 5 PAPA there so, 5 – box every time PAPA taking $2\log n$ & we selected $n - 1$ times
- ① $2\log n \times (n-1) \approx n \log n$
- ② Initially min-heap is not there so, will have to create that also take n time $\rightarrow$ using build heap
- So, finally: $\Rightarrow O(n) + O(n \log n) = O(n \log n)$

❖ Example(2):-

- ✓ Construct Huffman tree using these nodes?
- ✓ Find Huffman code for every Character?
- ✓ Find total no. of bits?
- ✓ Find average no. of bits?
- ✓ Find Time Complexity?

Value	A	B	C	D	E	F
Frequency	5	25	7	15	4	12

❖ Activity Selection Problem:-

- There are n different activities are given with their starting time and ending time.
- Select the maximum number of activities to solve by a single person.
- We will use the greedy approach to find the next activity whose finish time is minimum among rest activities.
- The start time is more than or equal with the finish time of the last selected activity.
- The complexity of this problem is $O(n \log n)$ when the list is not sorted.
- When the sorted list is provided the complexity will be $O(n)$.

❖ Example(1):-

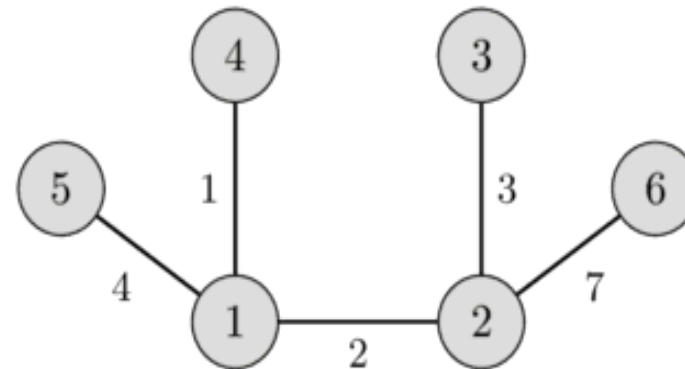
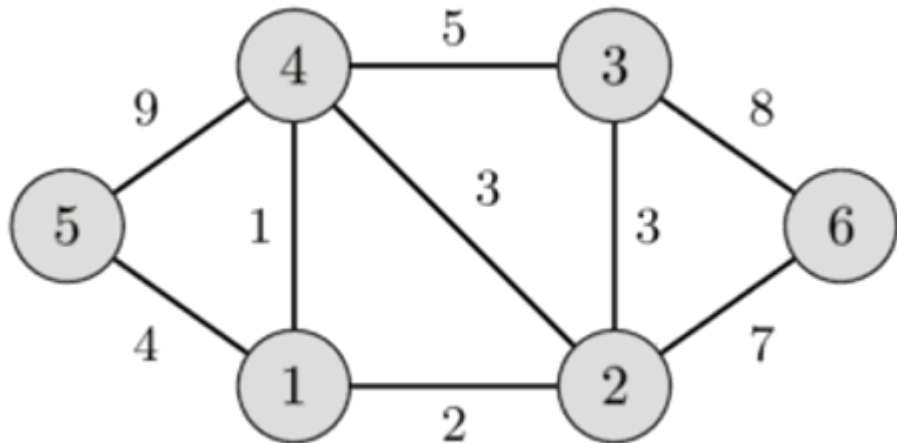
- Input
- A List of different activities with starting & ending times:
- $\{ (5,9), (1,2), (3,4), (0,6), (5,7), (8,9) \}$
- Output:
- Select Activities are:
- Activity: 0, Start:1 End:2
- Activity:1, Start:3 End:4
- Activity:3, Start:5 End:7
- Activity:5, Start:8 End:9

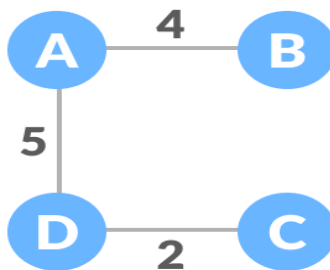
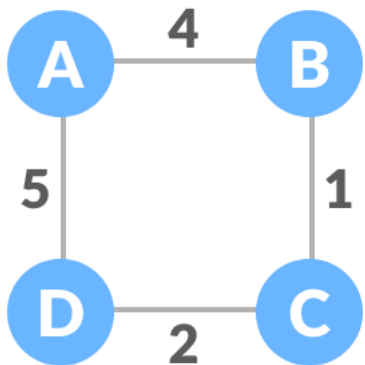
- ❖ Algorithm:-
 - Max Activity(act, size)
 - **Input:** A list of activity, and the number of elements in the list.
 - **Output** – The order of activities how they have been chosen.
- ☐ Begin
- ☐ initially sort the given activity List
- ☐ set $i := 1$
- ☐ display the i th activity //in this case it is the first activity
- ☐ for $j := 1$ to $n-1$ do
- ☐ if start time of $act[j] \geq$ end of $act[i]$ then
- ☐ display the j th activity
- ☐ $i := j$
- ☐ done
- ☐ End
- ❖ NOTE:-
- ❖ The [time complexity](#) of the activity selection problem using the greedy approach is **$O(n \log n)$** . If the completion time is given in sorted order then, the time complexity becomes **$O(n)$** .
- ❖ Example(2):-
- ❖ Explain and write an algorithm for greedy method of algorithm design. Given 10 activities along with their start and finish time as:
 $A = \{A1, A2, A3, A4, A5, A6, A7, A8, A9, A10\}$
 $S = \{1, 2, 3, 4, 7, 8, 9, 9, 11, 12\}$
 $F = \{3, 5, 4, 7, 10, 9, 11, 13, 12, 14\}$
Compute a schedule where the largest numbers of activities take place.

❖ Minimum Cost Spanning Tree:-

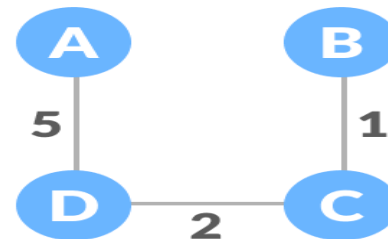
- A subgraph **S** of the given graph **G(V, E)** is said to be a **Spanning Tree** iff
 - S contains the **same number of vertices** as G
 - S contains **V - 1 edges** (where V is the number of vertices)
 - S is **acyclic** (contains **no cycles**)
- ◆ S is a **spanning tree** of G
- ◆ S is a **subgraph** of G that covers **all vertices** with **minimum number of edge** satisfying the spanning tree properties.

For Example:-

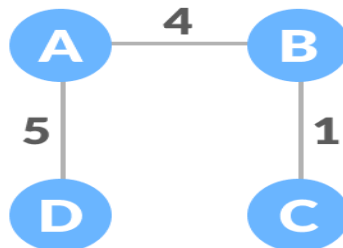




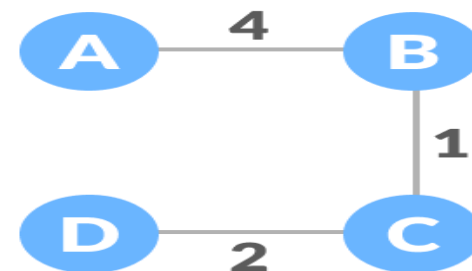
sum = 11



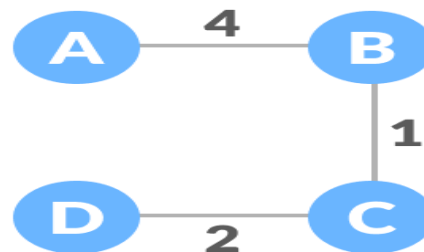
sum = 8



sum = 10



sum = 7



sum = 7

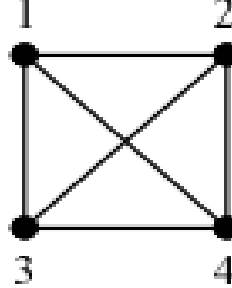


Figure 1: A four-vertex complete graph K_4 .

- The answer is 16. Figure 2 gives all 16 spanning trees of the four-vertex complete graph in Figure 1.
- Cayley devised the well-known formula n^{n-2} for the number of spanning trees in the complete graph K_n [1].
- Example:-
 - Find the total no. of Spanning Tree for Complete Graph K_5 .
 - Find the total no. of Spanning Tree for Complete Graph K_{10} .
 - Find the total no. of Spanning Tree for Two Complete Graph I .e K_4 & K_4 .

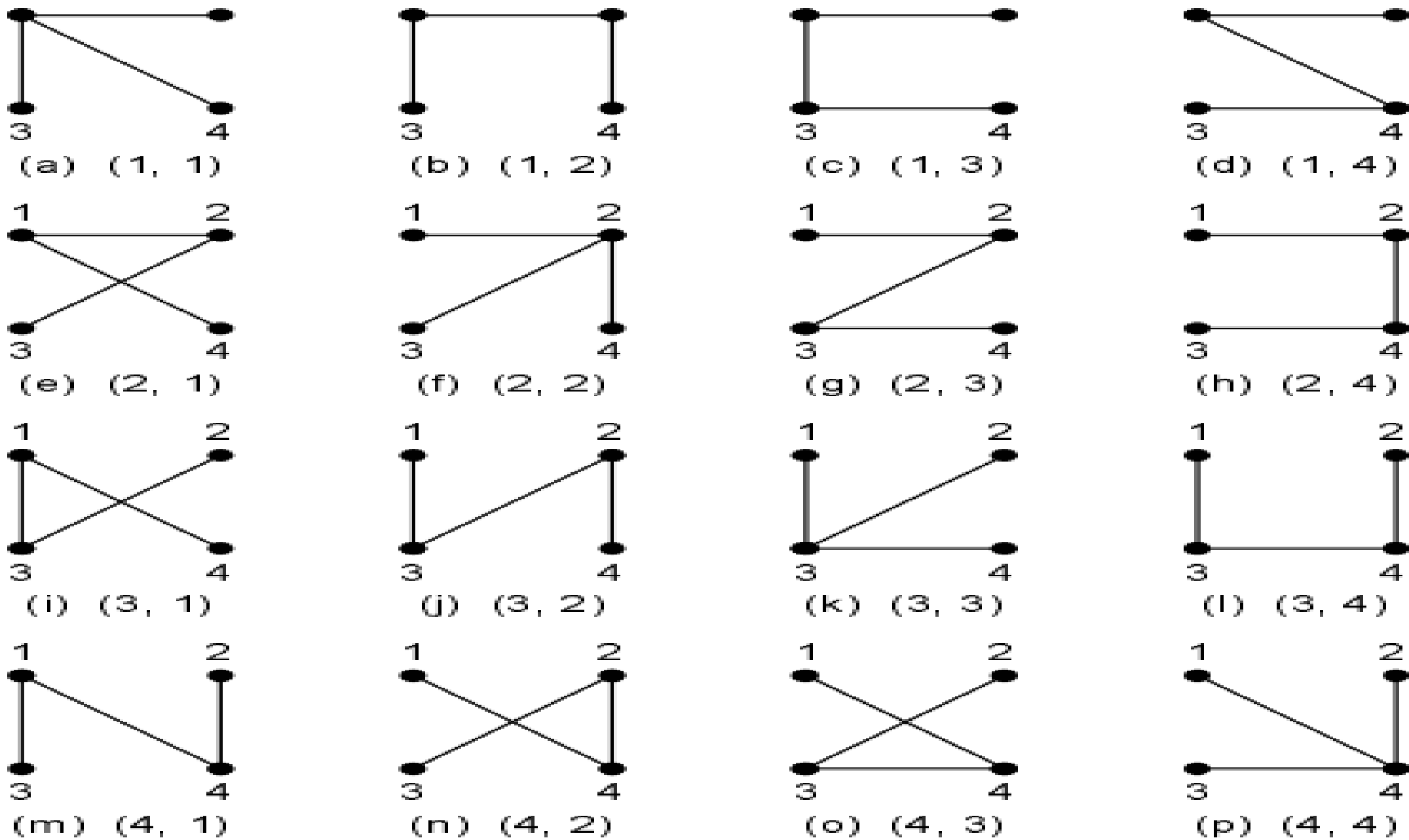
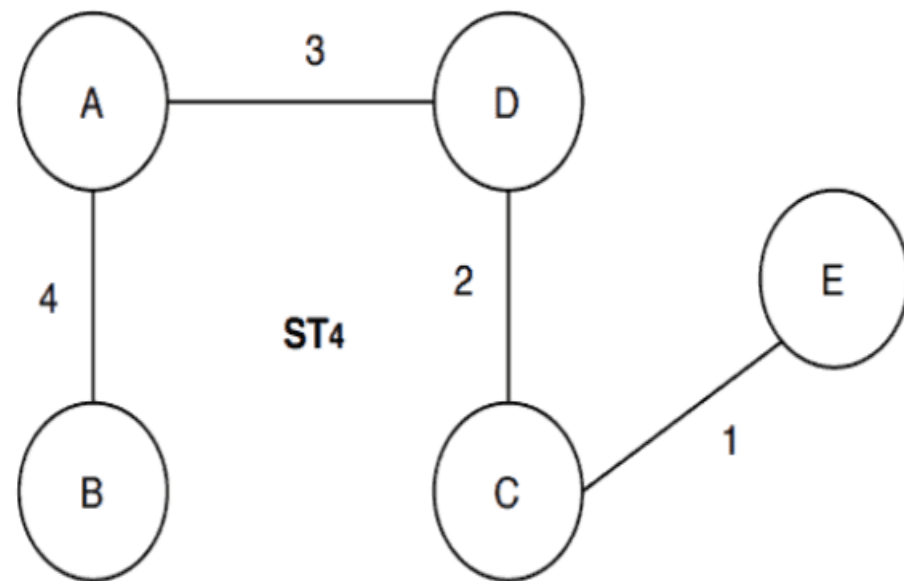
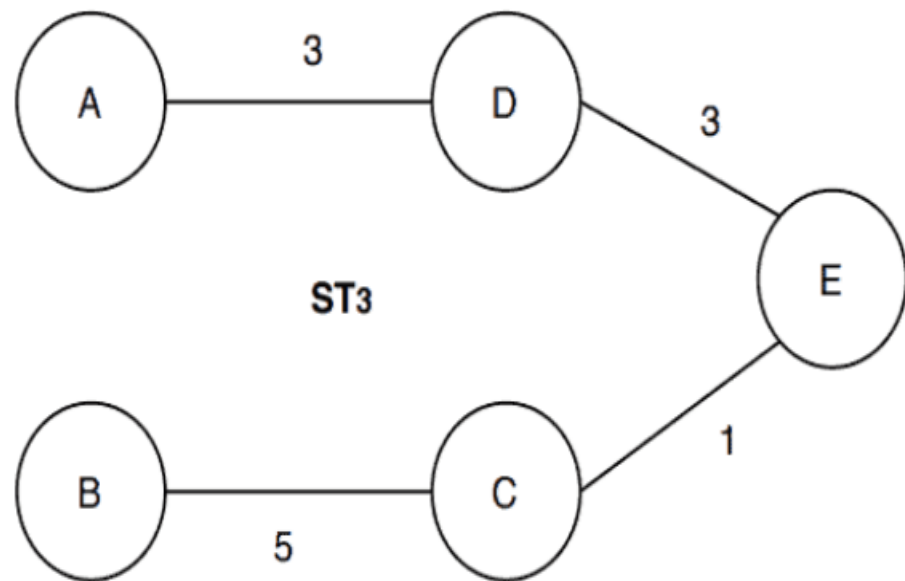
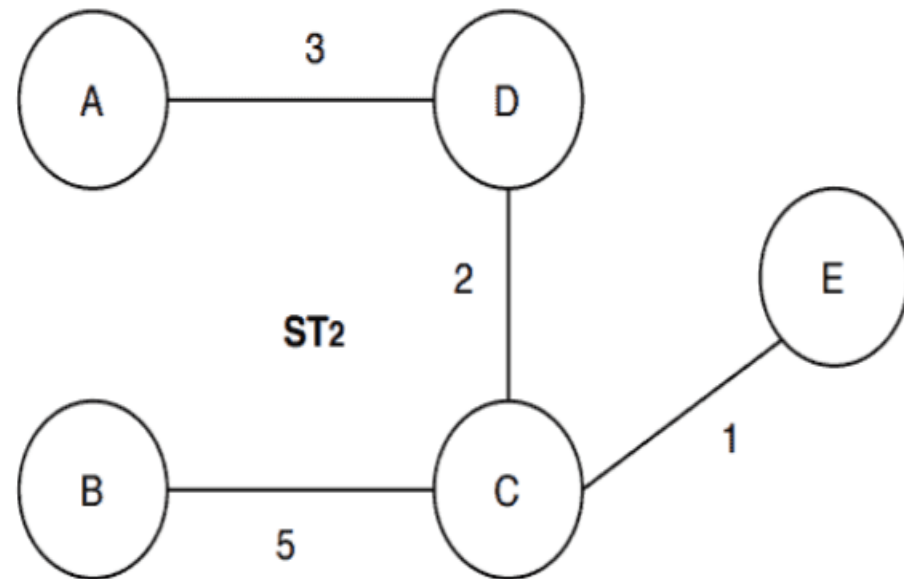
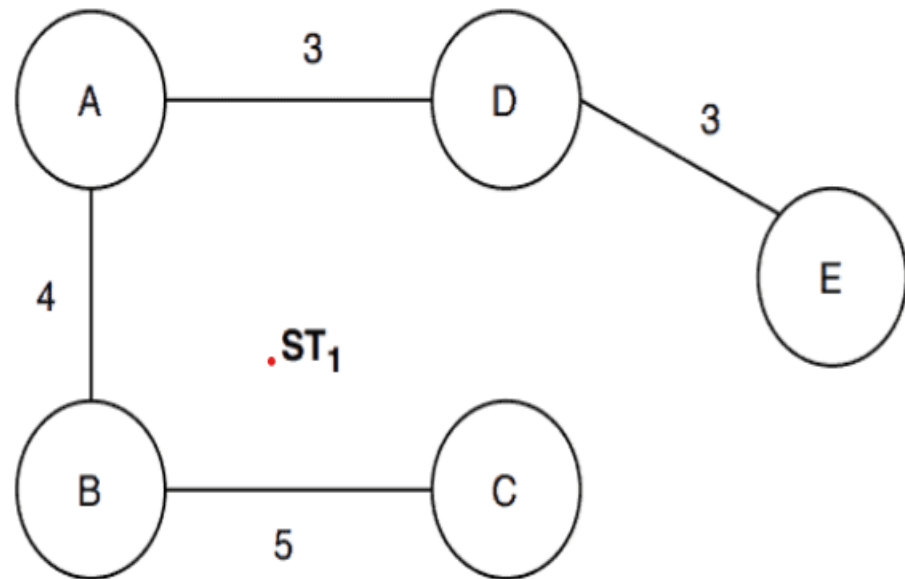
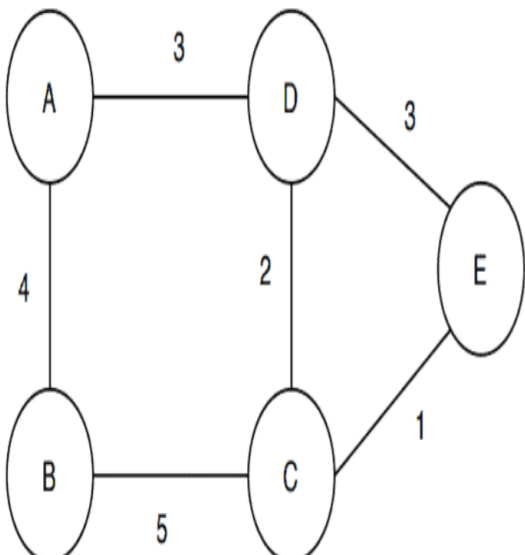
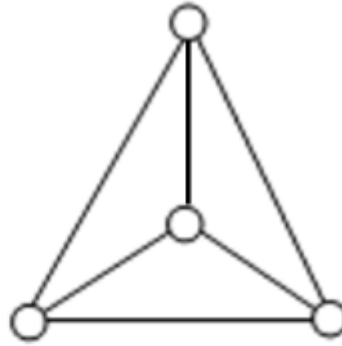


Figure 2: All 16 spanning trees of K_4 .



Example 18.1.10.

- Consider the graph at right. It is not a tree, because it has too many edges. We can make it a tree by removing some of the edges.



Some examples of spanning trees for this graph are shown below.

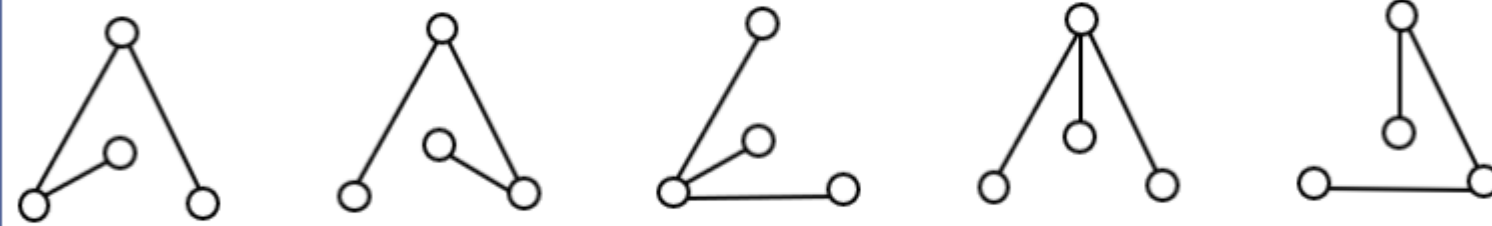
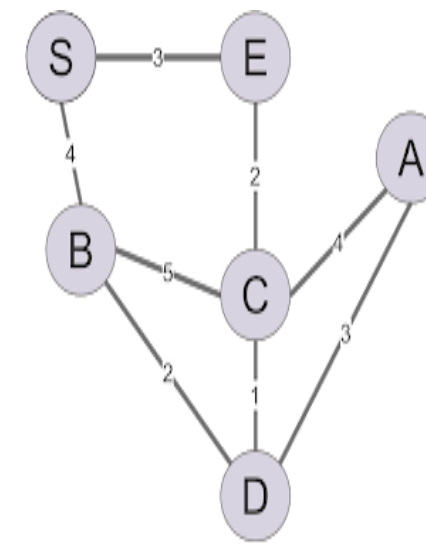


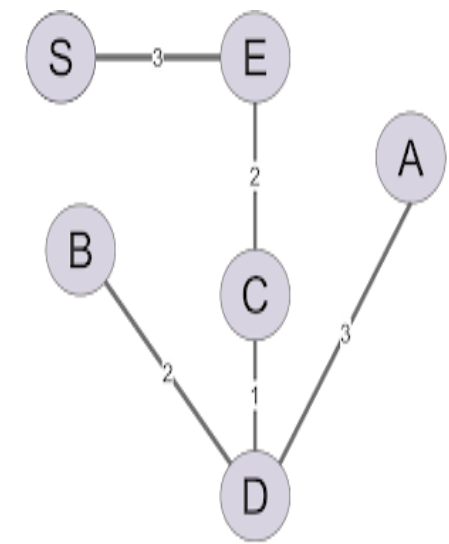
Figure 18.1.11.

Of course, any random spanning tree isn't really what we want. We want the **minimal spanning tree**.

Minimal spanning tree.
The **minimal spanning tree** (MST) is the spanning tree with the smallest total edge weight.



Connected Graph



Minimum Spanning Tree

Specific Graph Types:-

➤ Complete Graph (K_n):-

- A complete graph K_n has n vertices where every vertex is connected to every other vertex.
- The number of spanning trees for K_n is given by **Cayley's Formula**: $(K_n) = n^{n-2}$ for $n \geq 2$.
- For example:
 - K_2 (two vertices, one edge): $2^2 - 2 = 2^0 = 1$ spanning tree.
 - K_3 (triangle): $3^3 - 2 = 3^1 = 3$ spanning trees.
 - K_4 : $4^4 - 2 = 4^2 = 16$ spanning trees.

➤ Tree Graph:-

- If a graph G is itself a tree, then it has only **1** spanning tree (itself). By definition, a tree is a connected acyclic graph, so it's already a spanning tree of itself, and no other spanning tree can exist as adding or removing edges would violate the tree properties.

➤ Cycle Graph (C_n):-

- A cycle graph C_n has n vertices connected in a cycle
- The number of spanning trees for C_n is n . This is because to form a spanning tree from a cycle, you must remove exactly one edge. Since there are n edges in a cycle graph, there are n ways to choose which edge to remove

➤ Path Graph (P_n):-

- A path graph P_n has n vertices connected in a line.
- Similar to a tree, a path graph is already a tree itself. Therefore, it has only 1 spanning tree.

➤ Complete Bipartite Graph ($K_{m,n}$):-

- A complete bipartite graph $K_{m,n}$ has two sets of vertices,
- M with m vertices and N with n vertices, such that every vertex in M is connected to every vertex in N ,
- and there are no edges within M or N .
- The number of spanning trees for $K_{m,n}$ is: $(K_{m,n}) = m^{n-1} \cdot n^{m-1}$ for $m, n \geq 1$.

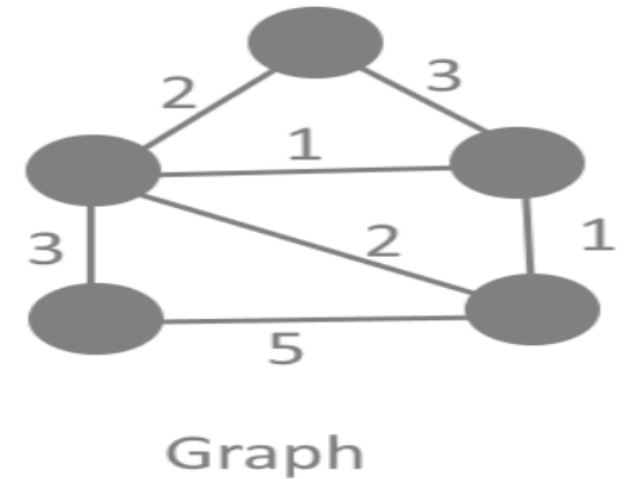
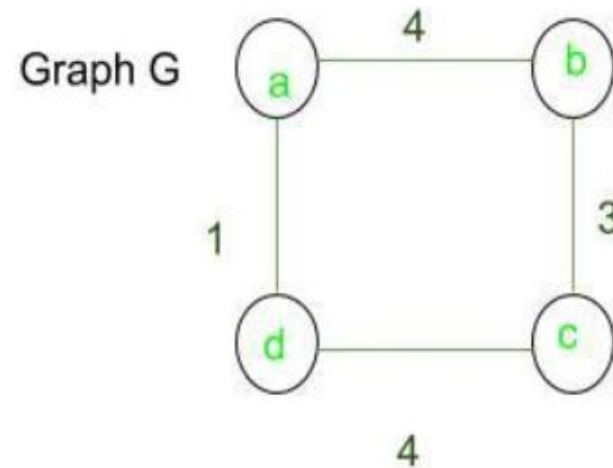
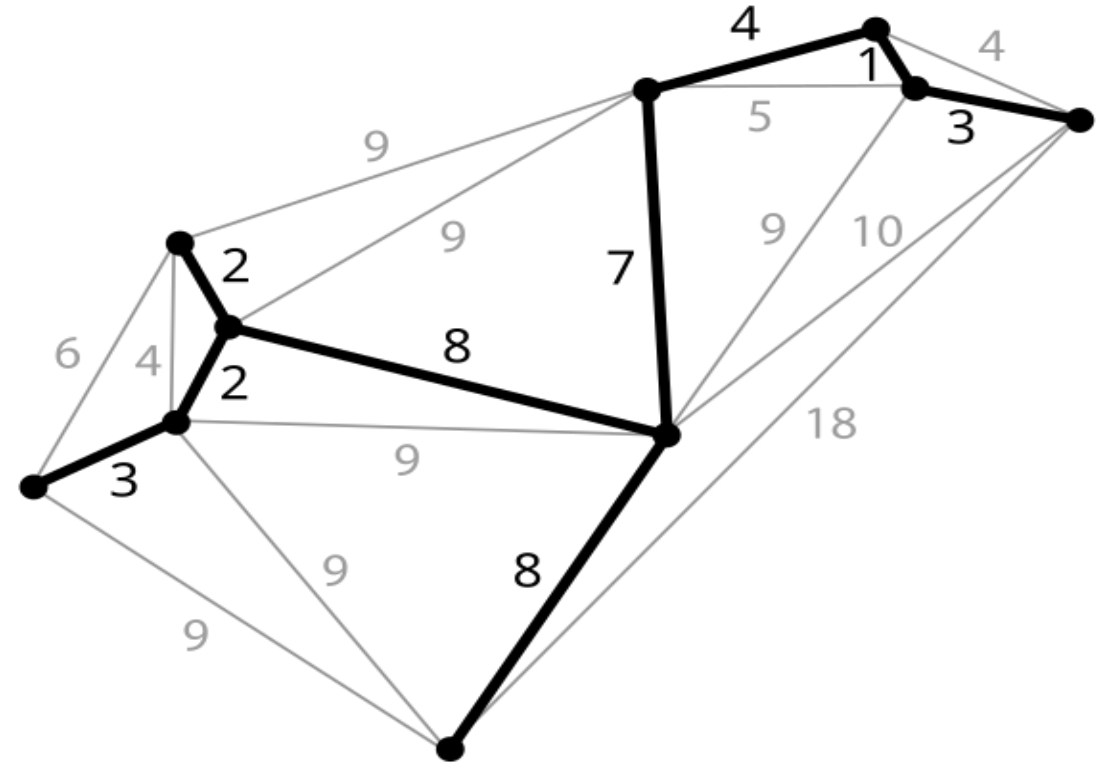
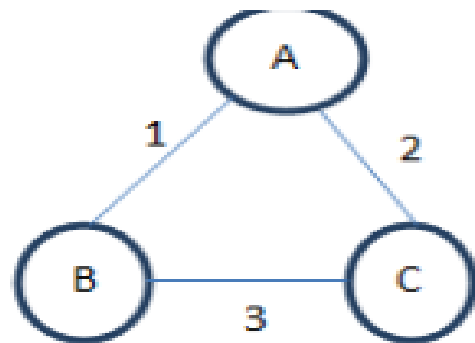
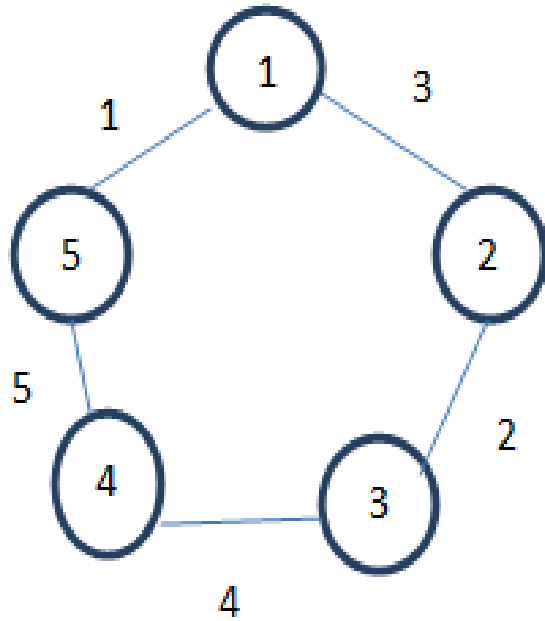
❖ Time Complexity Analysis:-

- In the case of a complete graph, the time complexity of the algorithm
- depends on the loop where we're calculating the sum of the edge weights of each spanning tree.
- The loop runs for all the vertices in the graph.
- Hence **the time complexity of the algorithm would be $O(v)$** .
- In case the given graph is not complete, we presented the matrix tree algorithm. Among all the operations, the most expensive one is finding the determining of the matrix. It takes $O(v^3)$ time.
- There fore, time complexity of algorithm would be $O(v^3)$.

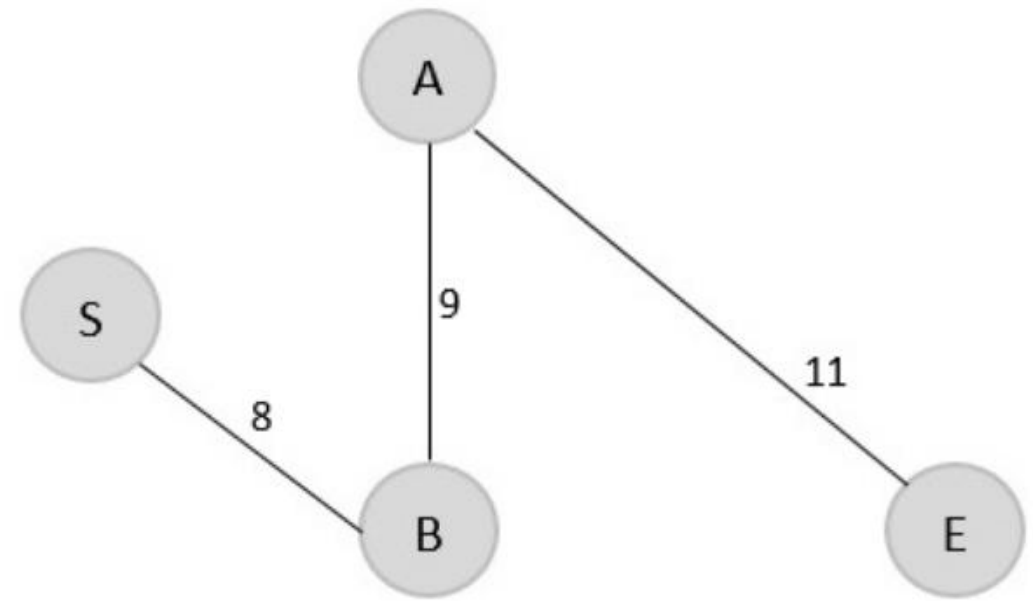
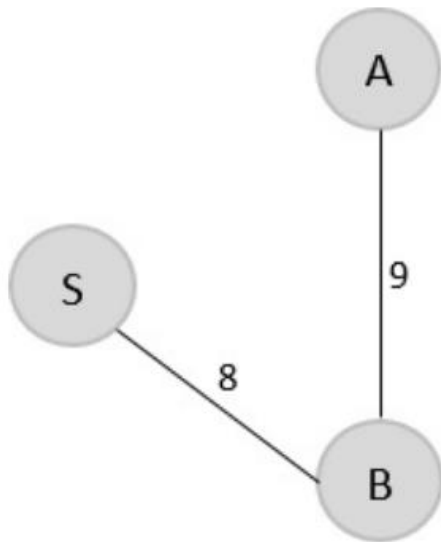
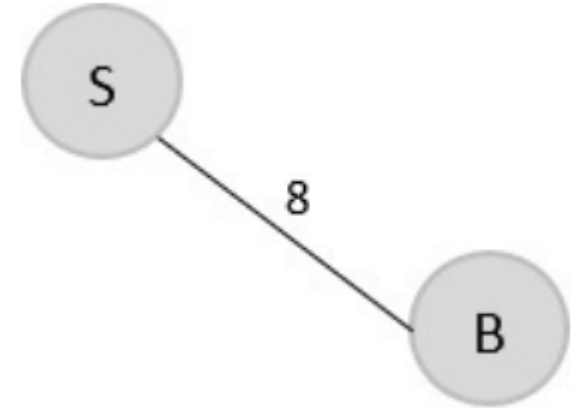
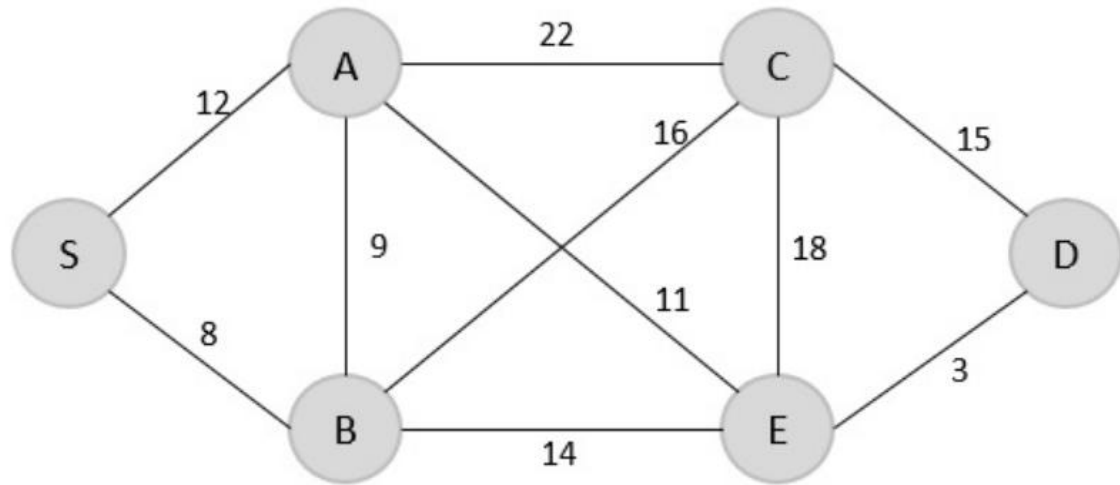
❖ Applications of MST:-

- An important application of the minimum spanning tree is to find the paths on the map.
- The minimum spanning tree is used to design networks like telecommunication networks, water supply networks, and electrical grids.
- Practical applications like cluster analysis, image segmentation, handwriting recognition all use the minimum spanning tree concept.

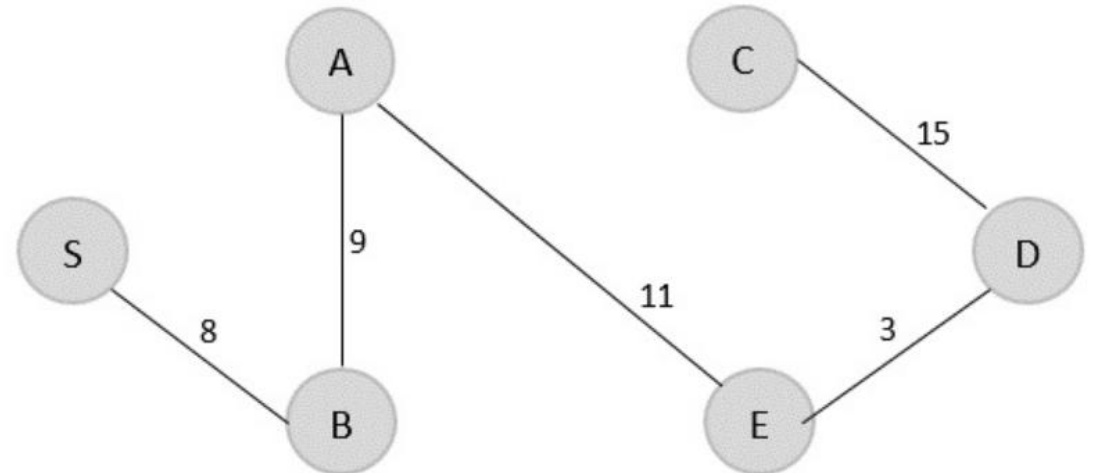
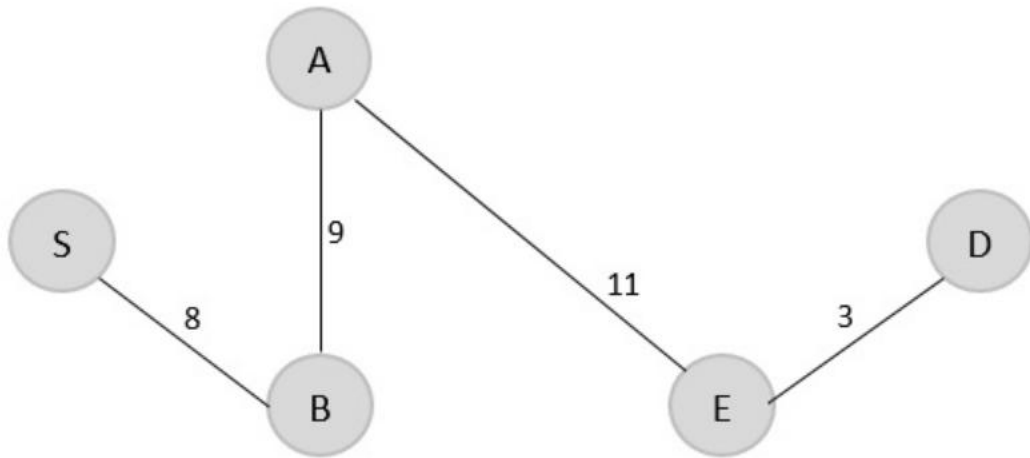
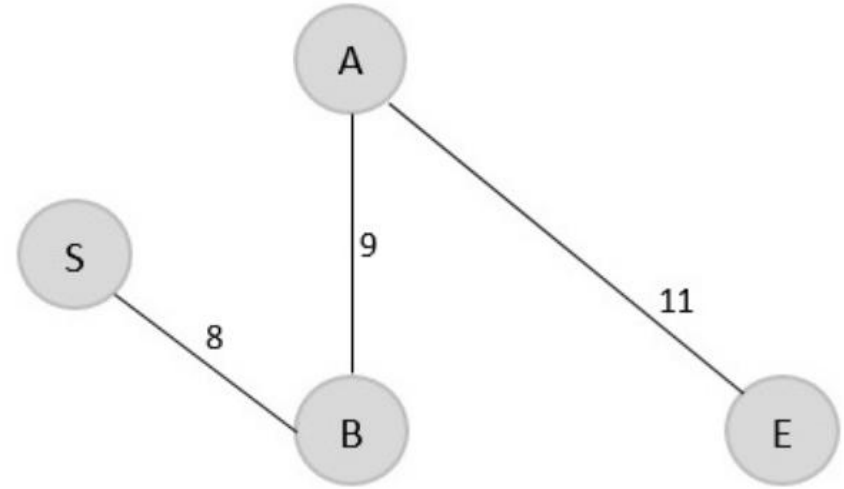
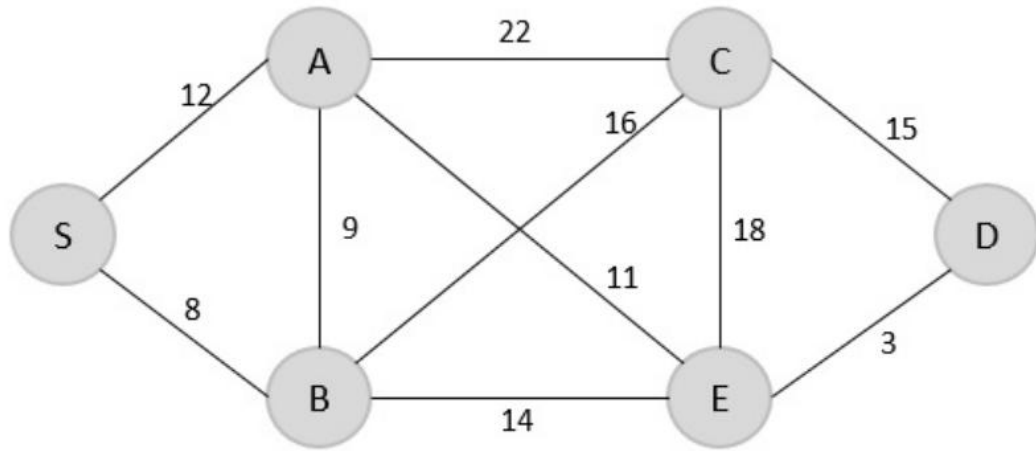
❖ Find the possible no. of spanning trees & MST for following graphs:-



Prims Algorithm



Prims Algorithm

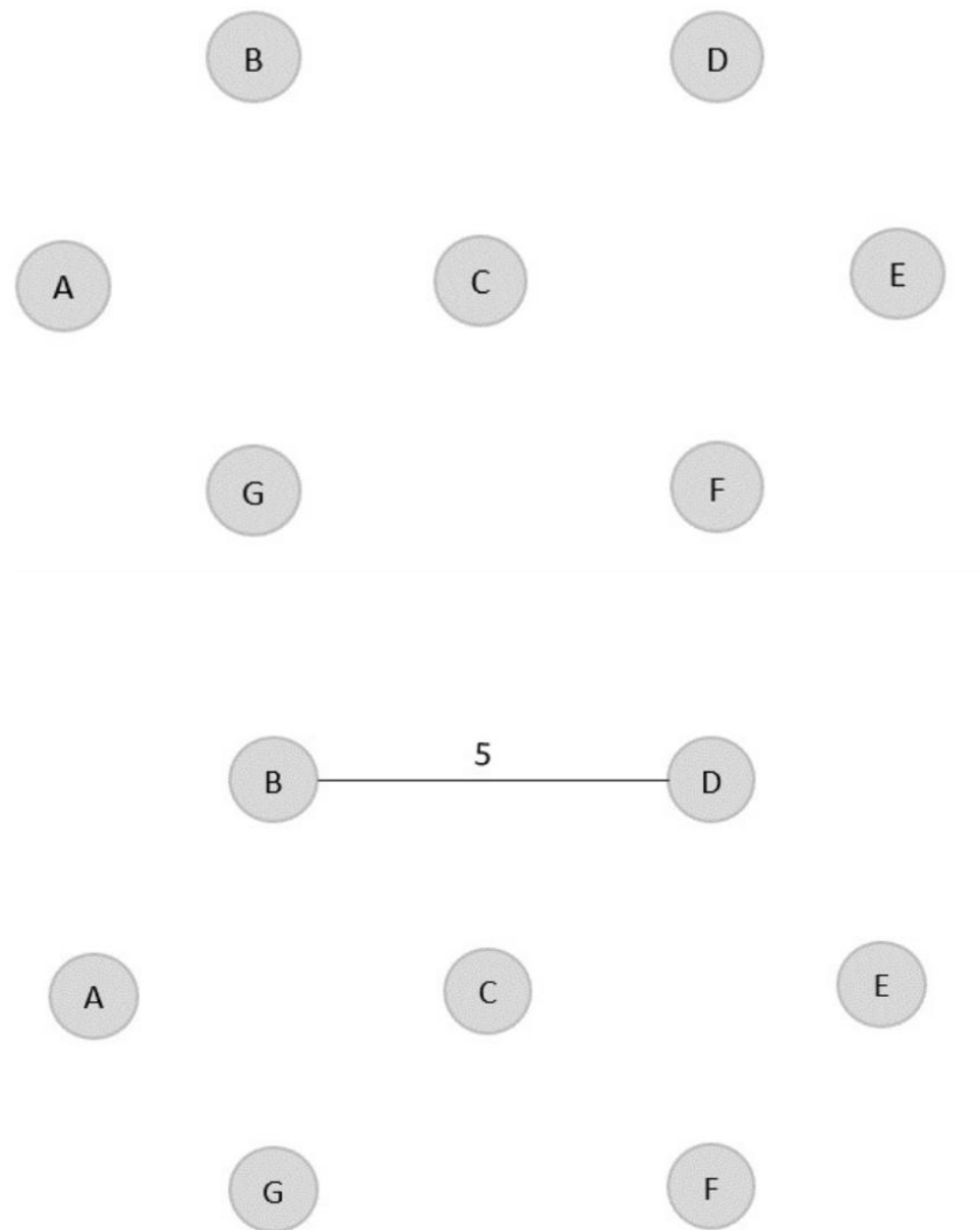
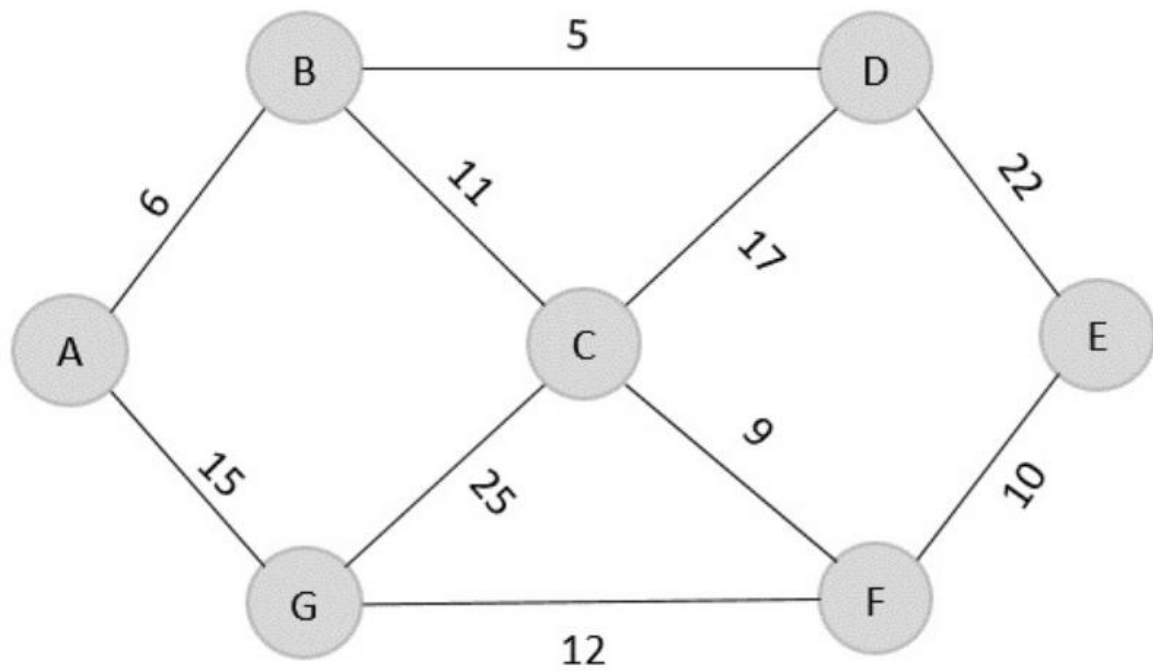


Prims Algorithm :

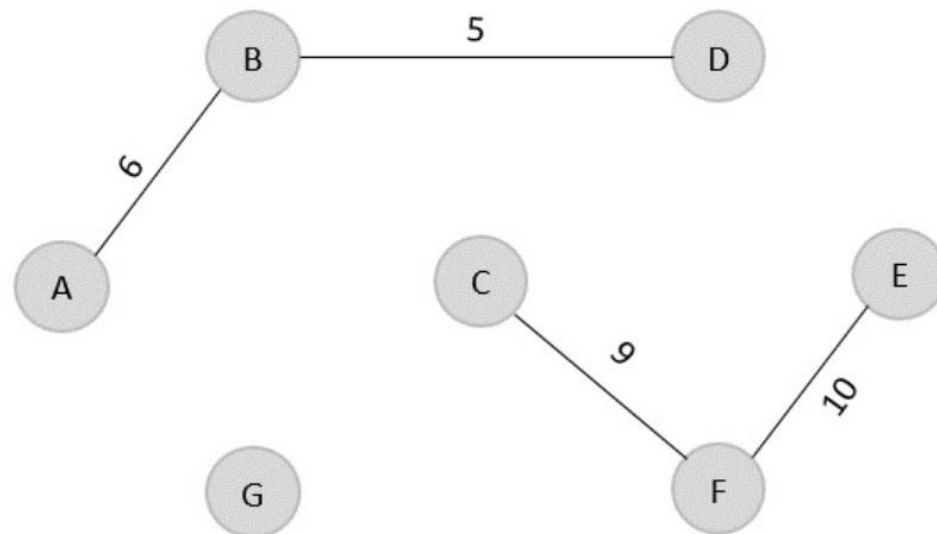
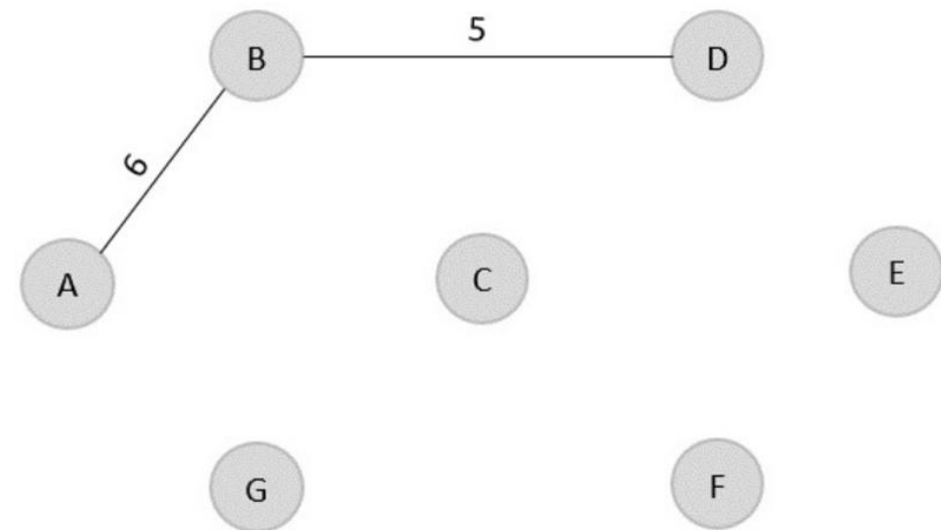
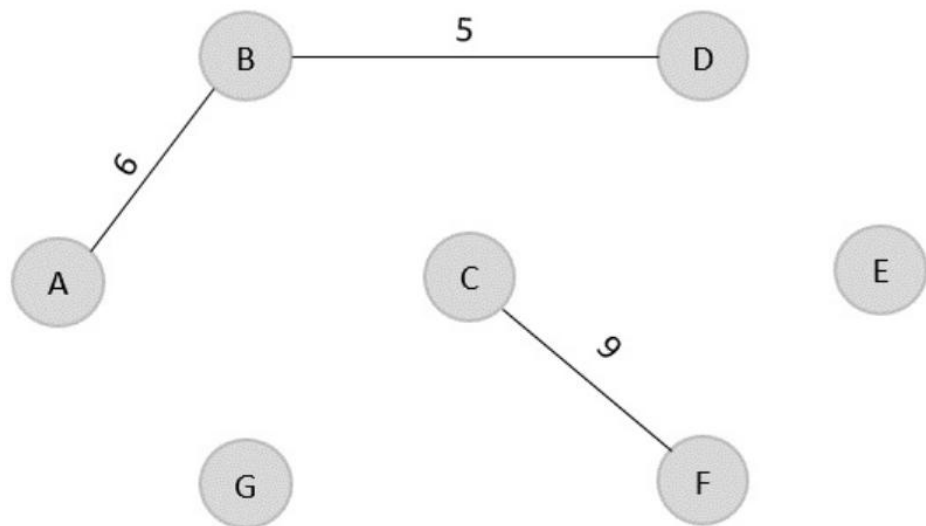
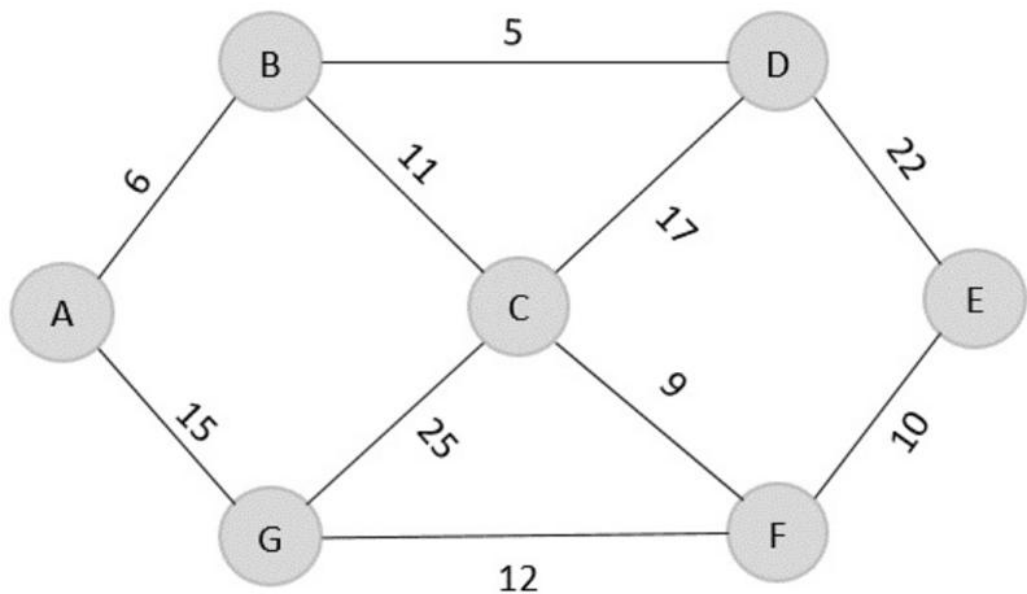
- 1.Start** with any node (say node 0).
- Create a **visited[]** array and mark the start node as visited.
- 3.Repeat until all nodes are visited:**
 - 1. From the **visited nodes**, find the **smallest edge** that connects to any **unvisited node**.
 - 2. Add that edge to the MST.
 - 3. Mark the new node as **visited**.
- 4.Repeat step 3** until all nodes are included.
- 5.Add up all the edge weights** to get the total cost.

Aspect	Complexity
Time Complexity	$O((V + E) \log V)$
Space Complexity	$O(V + E)$

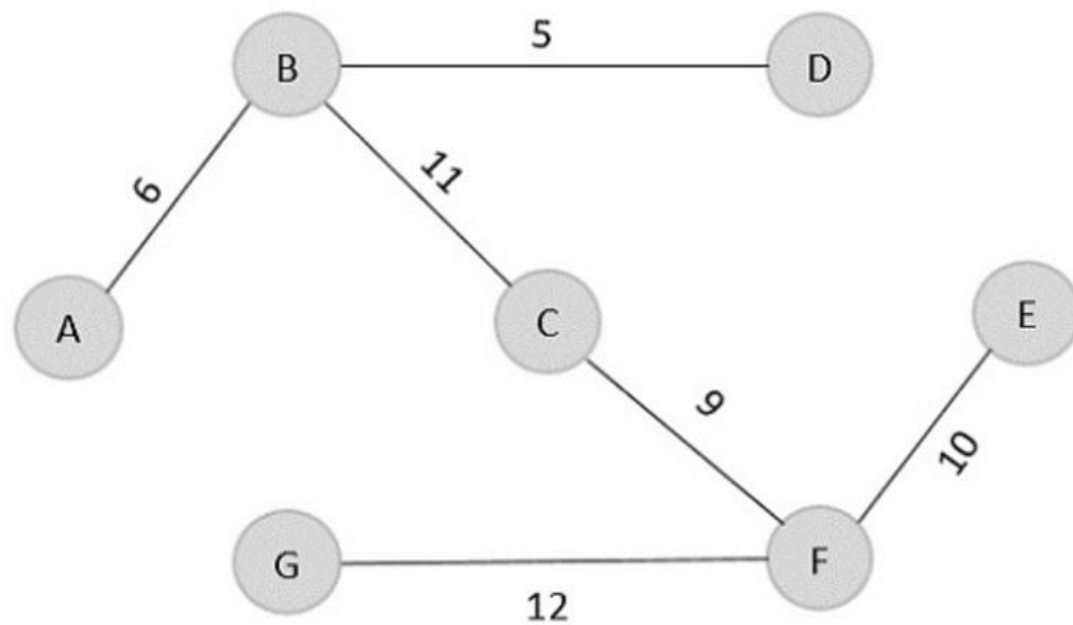
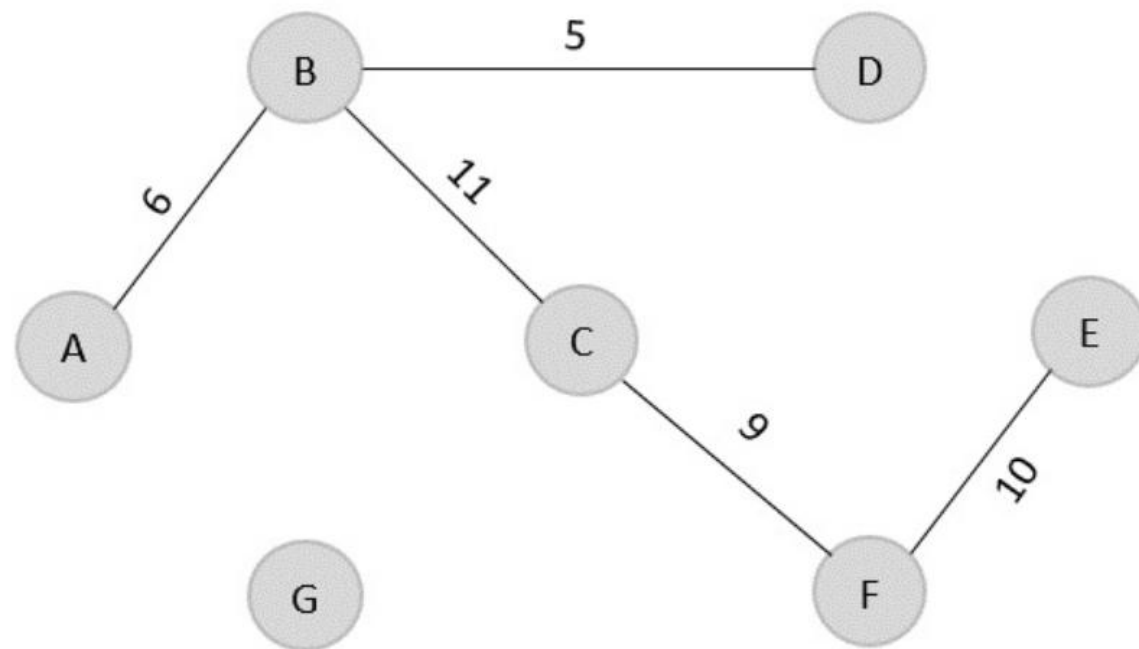
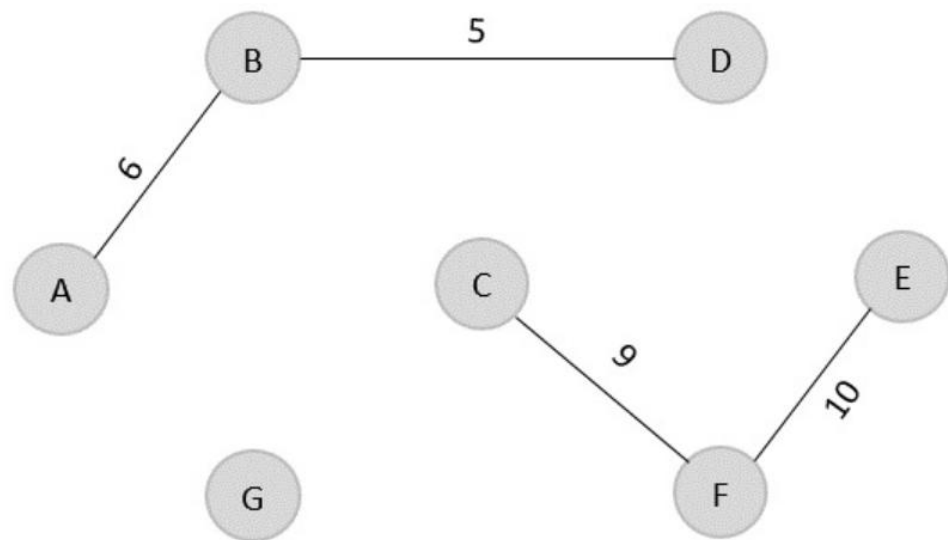
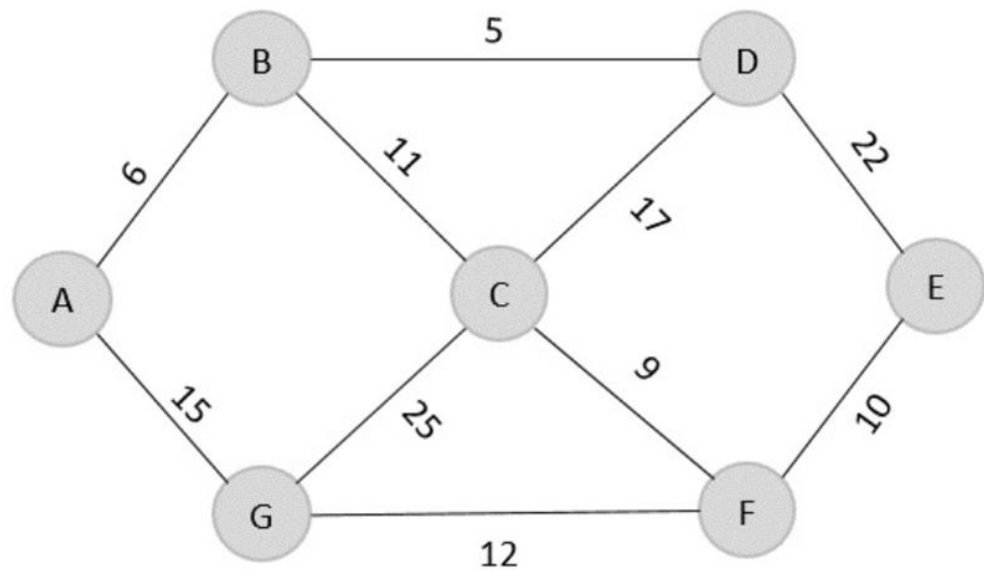
Kruskal's Algorithm



Kruskal's Algorithm



Kruskal's Algorithm

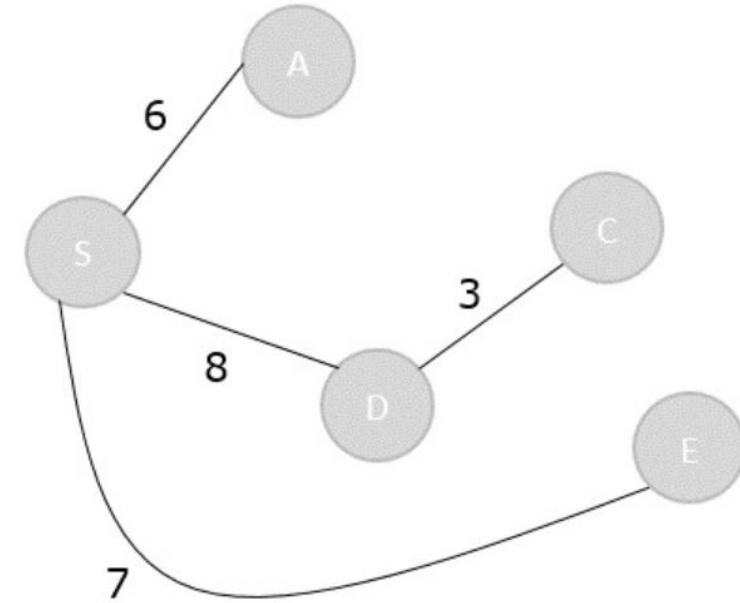
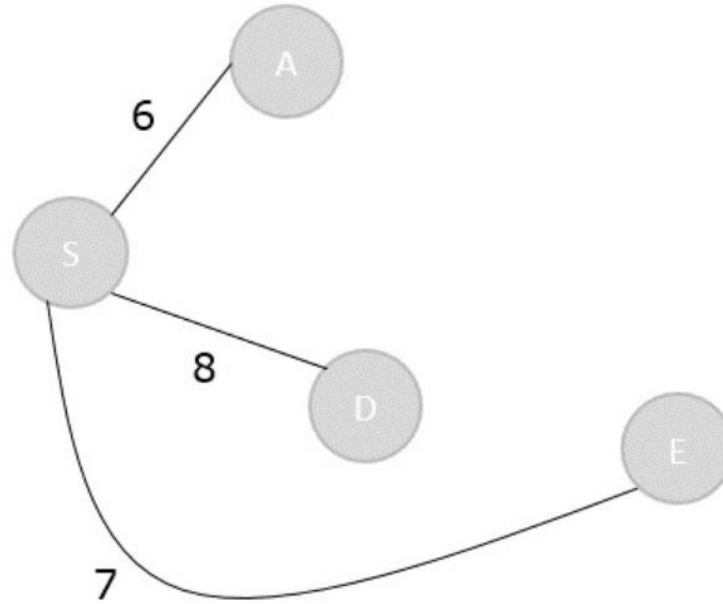
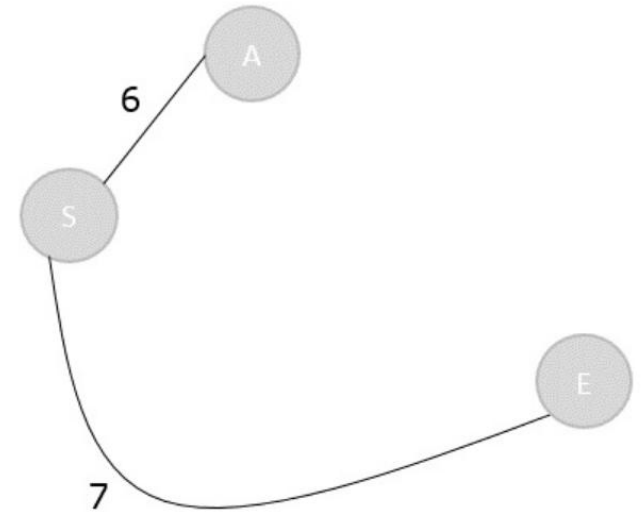
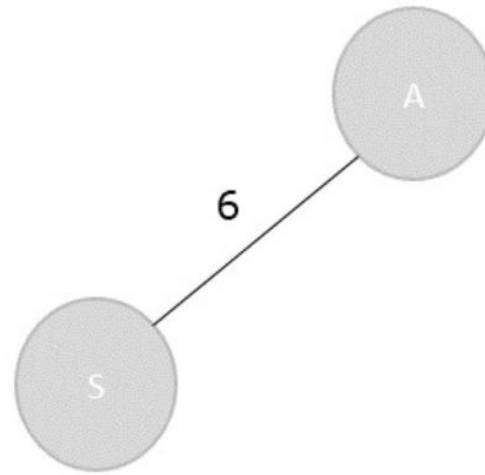
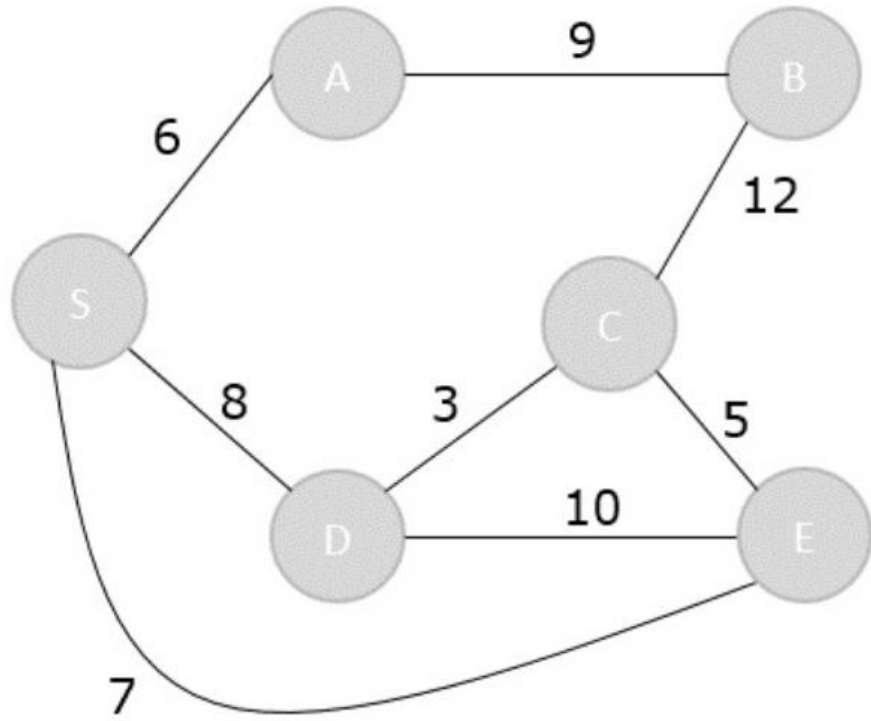


Kruskal's Algorithm :

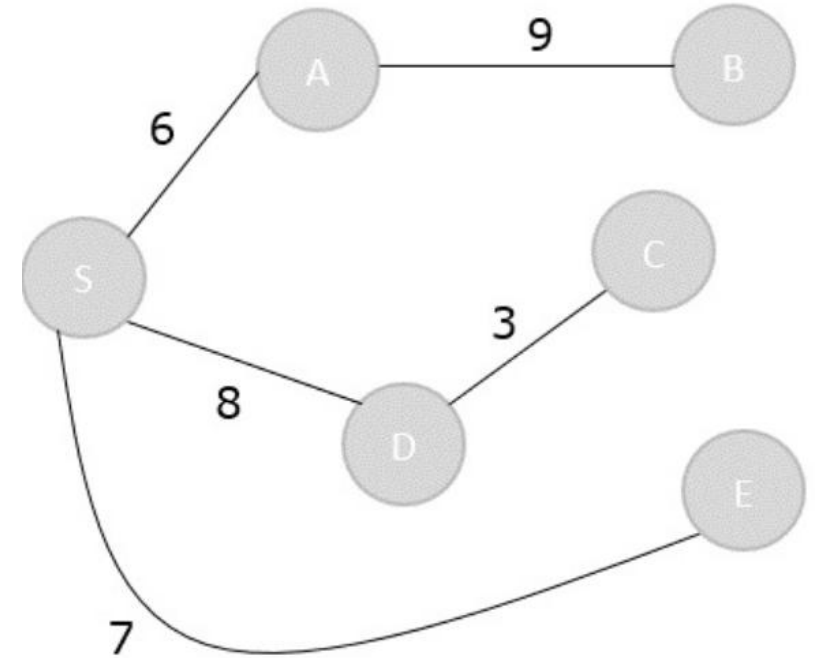
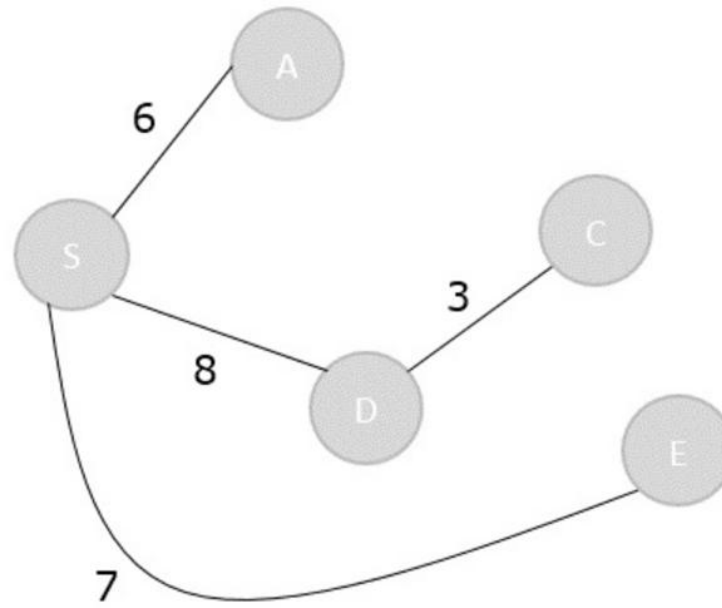
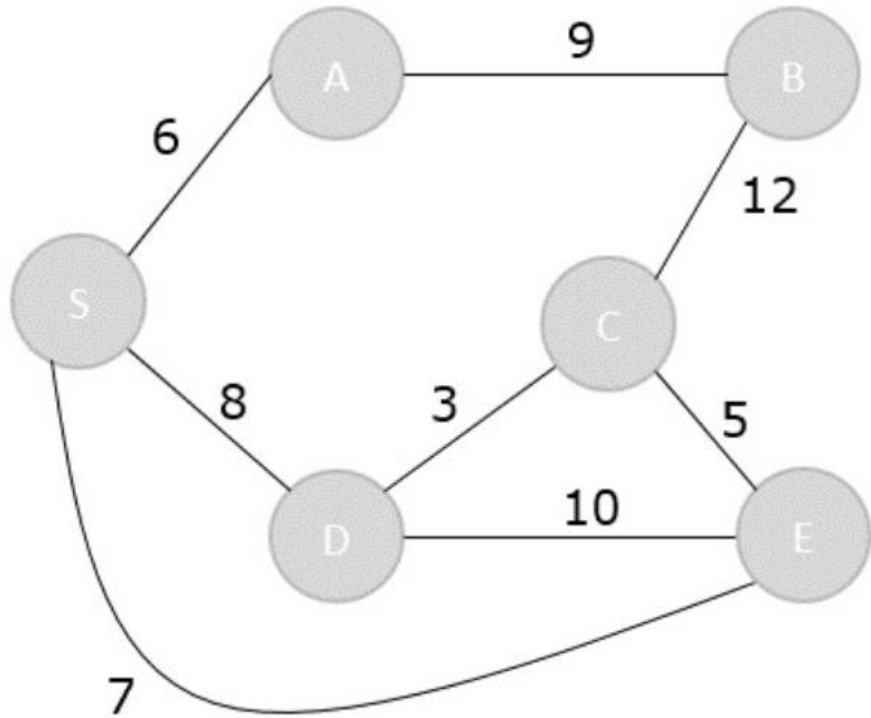
1. List all edges with weights.
2. Sort edges in increasing order of weight.
3. Initialize MST as empty.
4. For each edge (from smallest to largest):
 - If it connects two **unconnected nodes** (no cycle):
 - Add to MST.
 - Union the sets.
5. Repeat until MST has **(V - 1)** edges.
6. Add all selected edge weights → **Total MST cost.**

Complexity	Kruskal's Algorithm
Time Complexity	$O(E \log E)$
Space Complexity	$O(V + E)$

Dijkstras Algorithm



Dijkstras Algorithm



Dijkstra's Algorithm

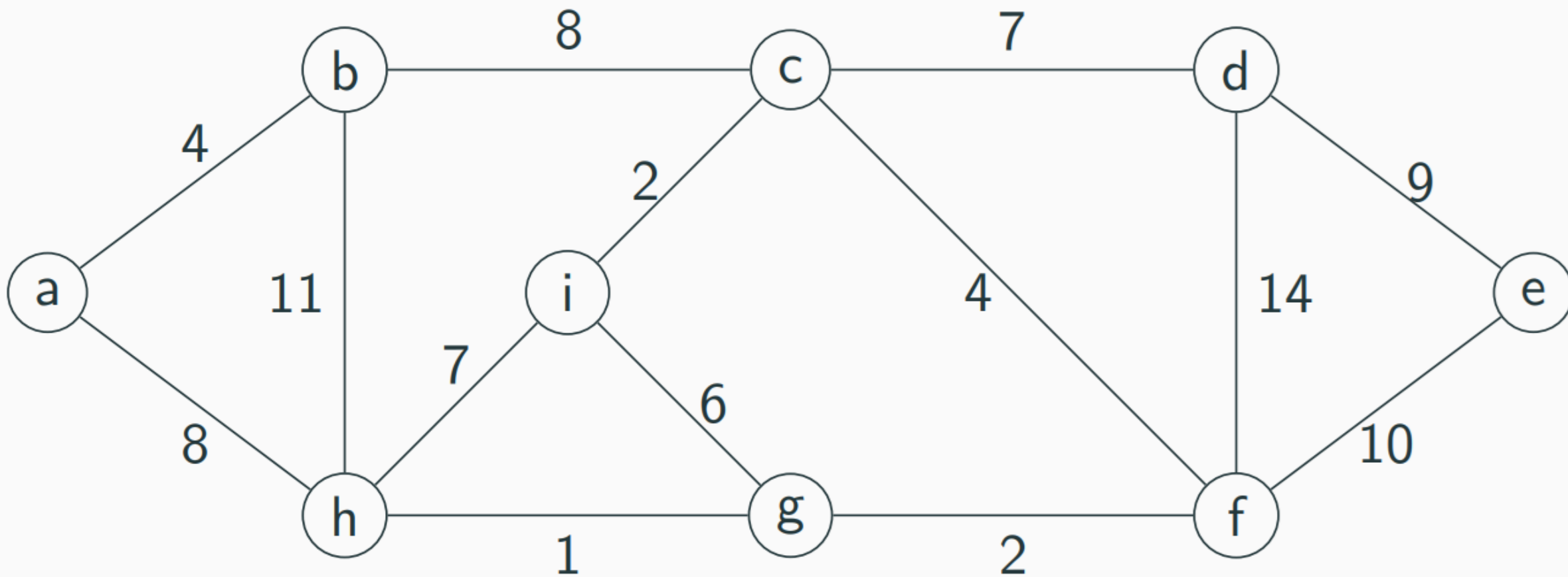
- 1. Create a `dist[]` array to store shortest distances.
 - Initialize all as **infinity**, except source = 0.
- 2. Create a `visited[]` array to track processed nodes.
 - All set to **false** initially.
- 3. Repeat for all nodes:
 - Pick the **unvisited node** with the **smallest distance**.
 - Mark it as **visited**.
 - For each **neighbor** of this node:
 - If not visited and path through this node is shorter, **update** the distance.
- 4. Repeat until all nodes are visited.

Aspect	Complexity
Time Complexity	$O((V + E) \log V)$
Space Complexity	$O(V)$

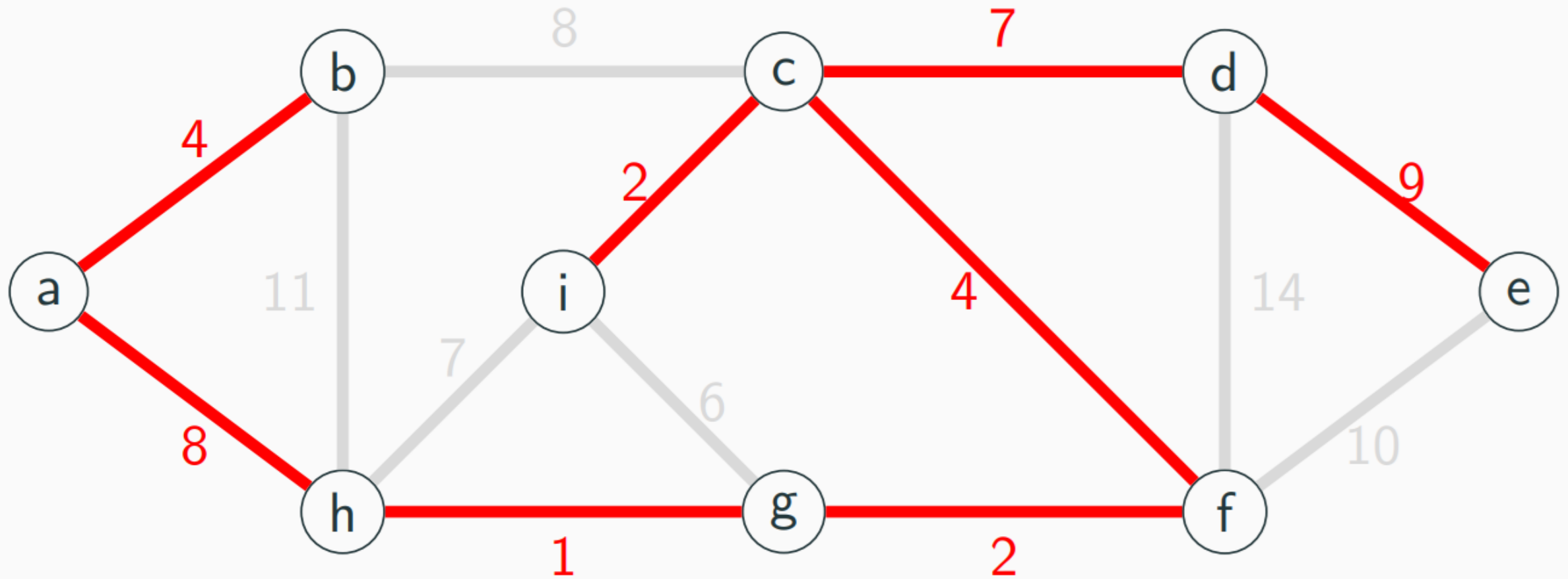
Feature	Prim's Algorithm	Kruskal's Algorithm	Dijkstra's Algorithm
Meaning	Builds MST starting from one node	Builds MST by choosing smallest edges first	Finds shortest path from one node to all others
Approach	Greedy – grow MST from a node	Greedy – pick smallest edges	Greedy – update shortest distance
Use Case (Real)	Designing electrical or computer networks	Connecting roads between cities with minimal cost	GPS, Google Maps, routing in networks
Can't Be Used	Can't find shortest paths between nodes	Can't find shortest paths	Can't be used if graph has negative weights
Works On	Undirected, connected, weighted graphs	Undirected, connected, weighted graphs	Directed or undirected graphs with non-negative weights
Output	Minimum Spanning Tree	Minimum Spanning Tree	Shortest path distances from source

GREEDY APPROACH	DIVIDE AND CONQUER
1.Many decisions and sequences are guaranteed and all the overlapping subinstances are considered.	1.Divide the given problem into many subproblems. Find the individual solutions and combine them to get the solution for the main problem
2. Follows Bottom-up technique	2. Follows top down technique
3.Split the input at every possible point rather than at a particular point	3.Split the input only at specific points (midpoint), each problem is independent.
4. Sub problems are dependent on the main Problem	4. Sub problems are independent on the main Problem
5. Time taken by this approach is not that much efficient when compared with DAC.	5. Time taken by this approach efficient when compared with GA.
6.Space requirement is less when compared DAC approach.	6.Space requirement is very much high when compared GA approach.

Kruskal's Algorithm

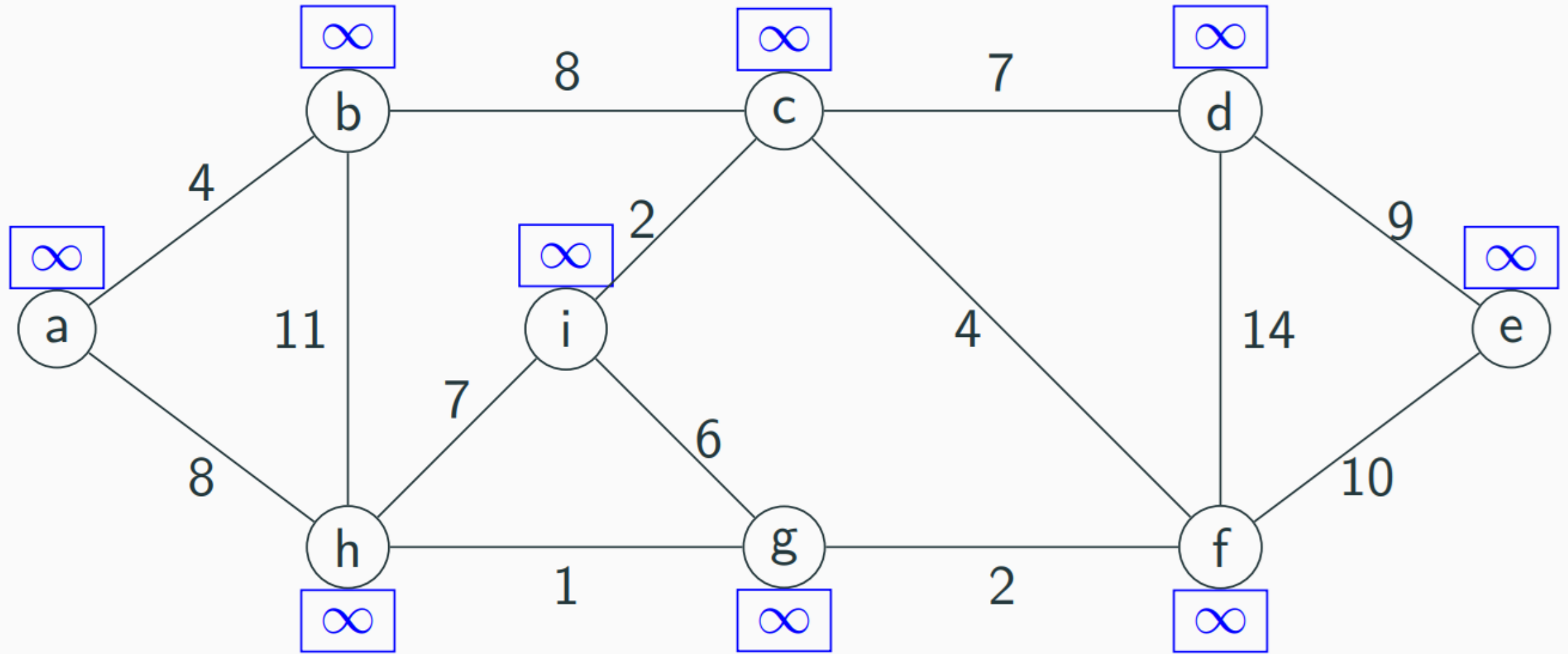


Kruskal's Algorithm

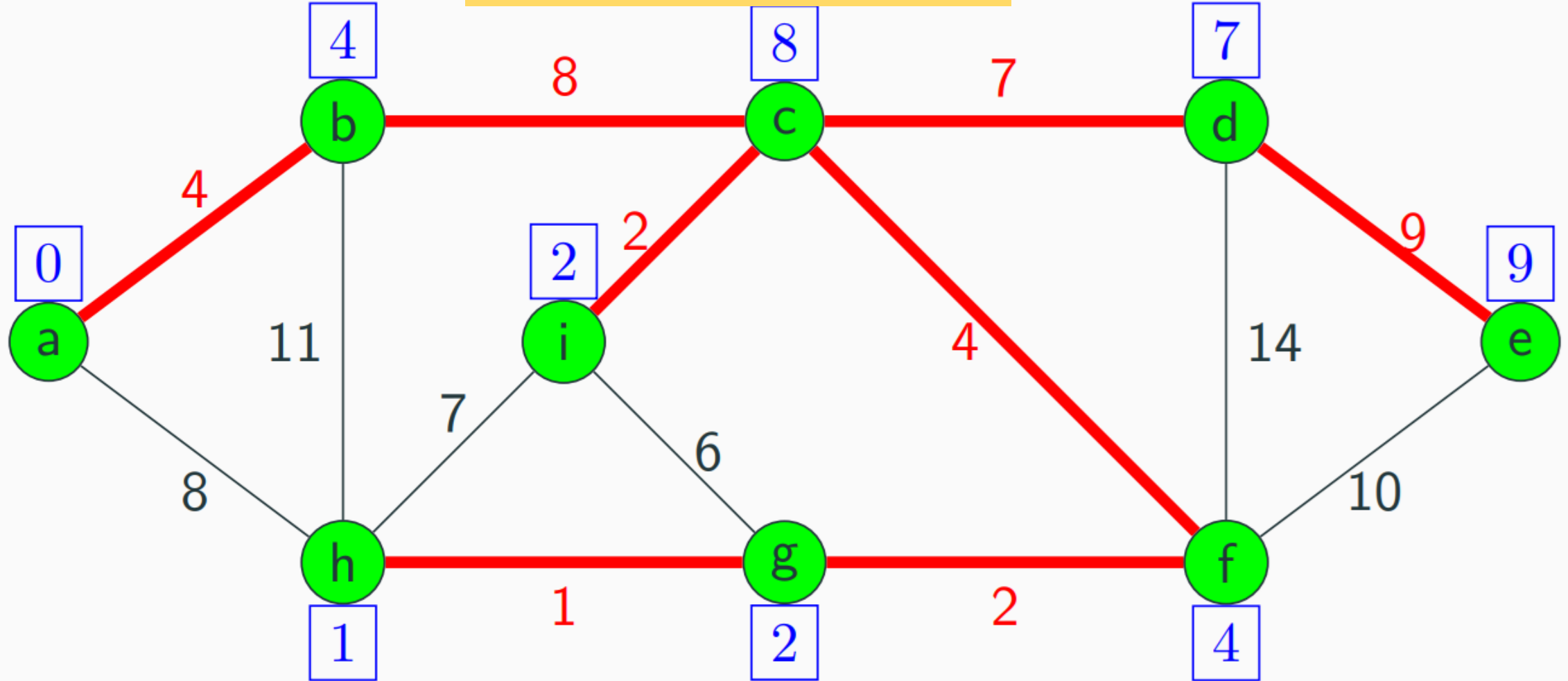


Answer

Prims Algorithm

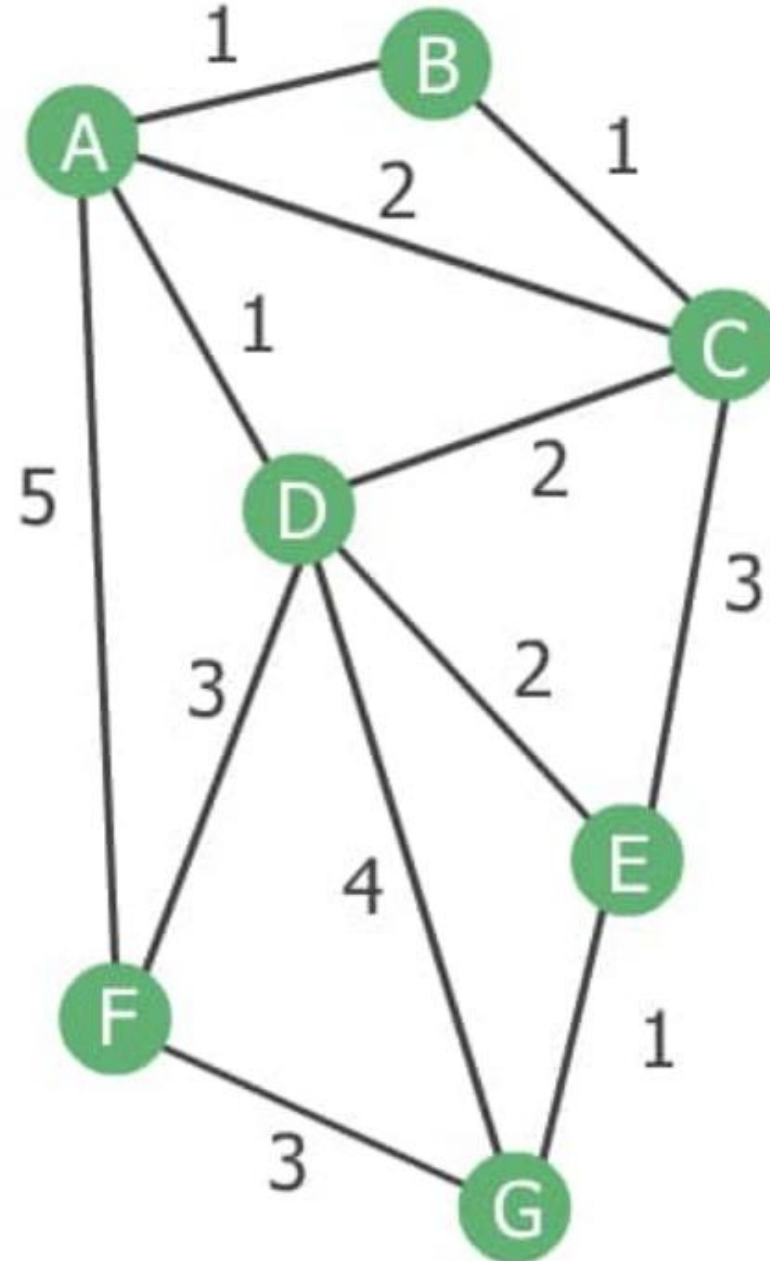


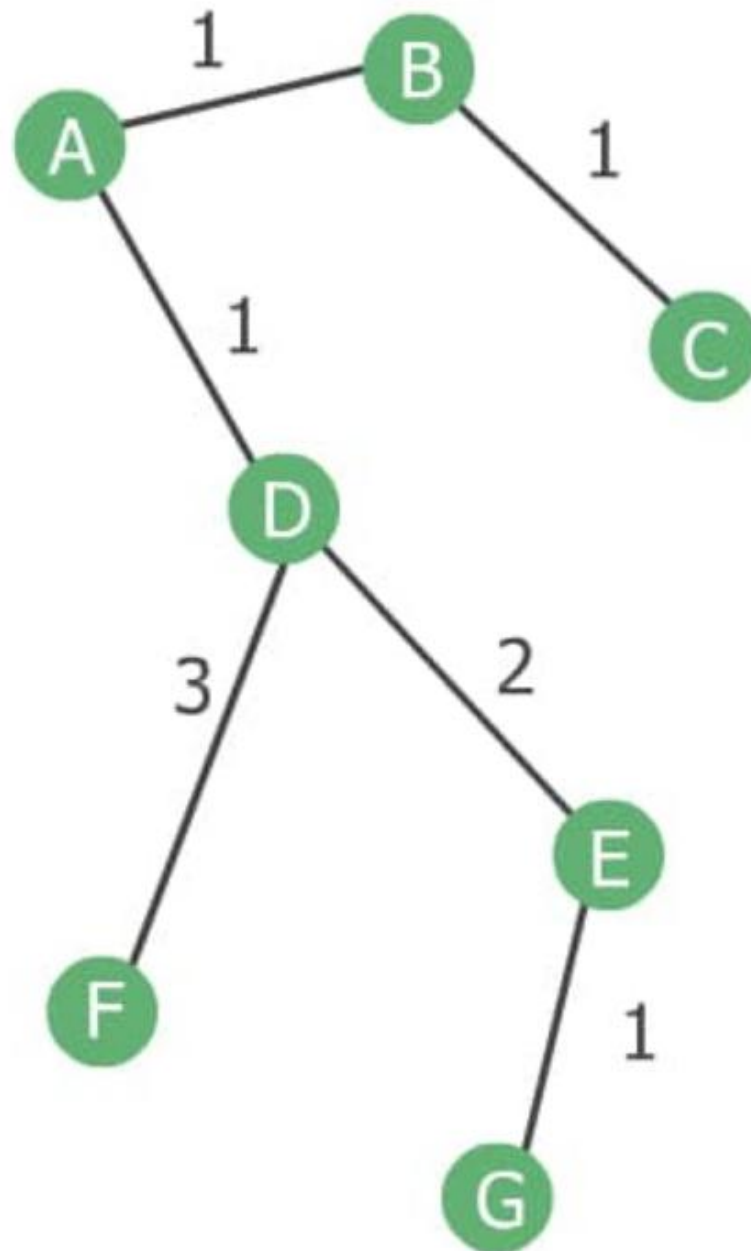
Prims Algorithm



Answer

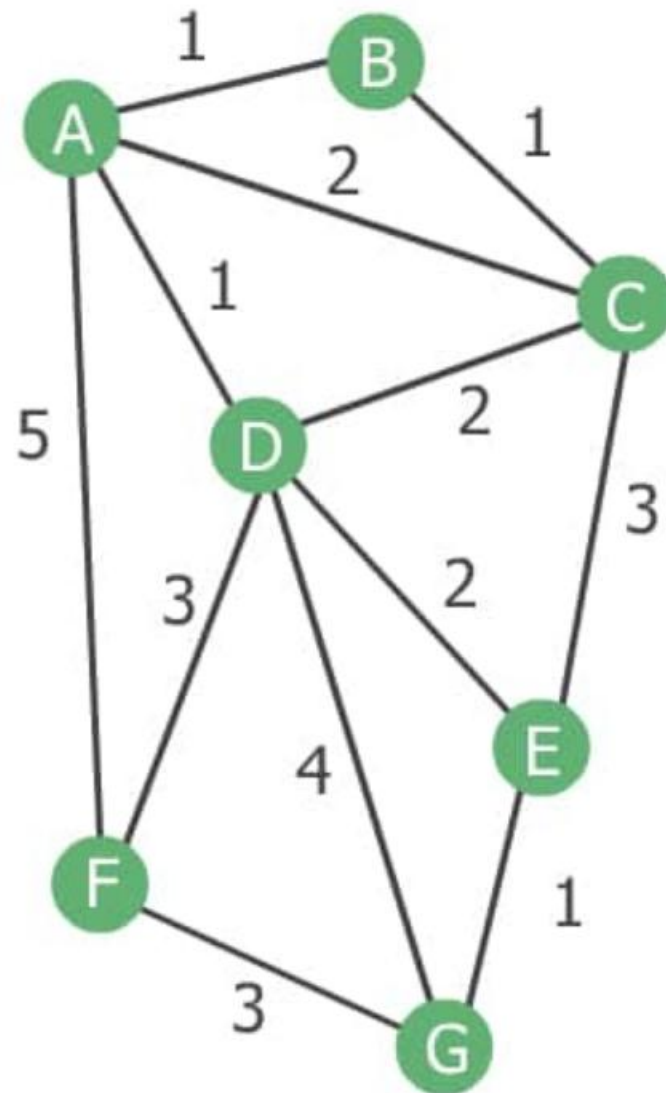
Minimum Spanning Tree (MST)



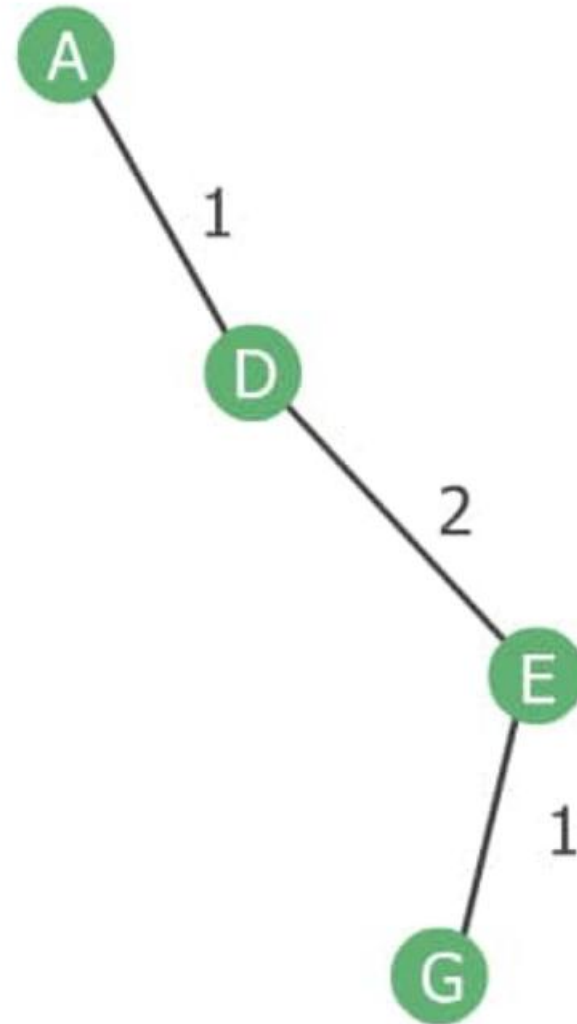


Answer

Shortest Path A to G



Original Graph



Shortest Path From A to G

Answer